



دانشگاه علم و صنعت
دانشکده مهندسی برق

عنوان:

هوش مصنوعی توضیح پذیر

نام نویسنده

محمد مهماندوست

نیم سال دوم ۱۴۰۴-۱۴۰۵

سلام افلا

فهرست مطالب

۱	مقدمه	۱
۱-۱	یادگیری ماشین چیست؟	۱
۲-۱	فرآیند ۳ مرحله‌ای یادگیری ماشین	۱
۳-۱	مزایا و معایب یادگیری ماشین	۲
۴-۱	اصطلاحات تخصصی	۲
۲	تفسیرپذیری	۴
۱-۲	اهمیت تفسیرپذیری	۴
۲-۲	چه زمانی به تفسیرپذیری نیاز نداریم؟	۵
۳-۲	طبقه‌بندی روش‌های تفسیرپذیری	۵
۴-۲	محدوده تفسیرپذیری	۵
۵-۲	ارزیابی تفسیرپذیری	۶
۶-۲	ویژگی‌های توضیحات	۶
۳	مدل‌های تفسیرپذیر	۸
۱-۳	رگرسیون خطی (Linear Regression)	۸
۱-۱-۳	مفروضات مدل رگرسیون خطی	۹
۲-۱-۳	تفسیر ضرایب و معیار ارزیابی	۹
۳-۱-۳	اهمیت ویژگی‌ها و تفسیر بصری	۱۰
۲-۳	رگرسیون لجستیک (Logistic Regression)	۱۰
۱-۲-۳	چرا رگرسیون خطی برای طبقه‌بندی مناسب نیست؟	۱۱
۲-۲-۳	تئوری مدل	۱۲
۳-۲-۳	تفسیر ضرایب	۱۲
۴-۲-۳	مزایا و معایب	۱۳
۳-۳	درخت تصمیم (Decision Tree)	۱۳
۱-۳-۳	تفسیر و اهمیت ویژگی‌ها	۱۴
۲-۳-۳	تجزیه درخت	۱۴

۱۵	 مزایا و معایب درخت تصمیم	۳-۳-۳
۱۵	 قوانین تصمیم (Decision Rules)	۴-۳
۱۵	 ساختار و مفاهیم پایه	۱-۴-۳
۱۶	 استراتژی‌های ترکیب قوانین	۲-۴-۳
۱۶	 الگوریتم‌های یادگیری قوانین	۳-۴-۳
۱۷	 مزایا و معایب قوانین تصمیم	۴-۴-۳
۱۷	 سایر مدل‌های تفسیرپذیر	۵-۳
۱۸	 طبقه‌بند بیز ساده (Naive Bayes Classifier)	۱-۵-۳
۱۸	 روش k همسایه نزدیک (K-Nearest Neighbors / KNN)	۲-۵-۳
۲۰	 روش‌های مستقل از مدل تفسیرپذیری (Model-Agnostic Methods)	۴
۲۰	 نمودار وابستگی جزئی (PDP)	۱-۴
۲۱	 ویژگی‌های رسته‌ای و مثال‌های کاربردی	۱-۱-۴
۲۴	 مزایا و معایب	۲-۱-۴
۲۵	 مدل جایگزین سراسری (Global Surrogate)	۲-۴
۲۵	 مبانی نظری و ریاضیات	۱-۲-۴
۲۵	 مراحل ساخت و ارزیابی مدل جایگزین	۲-۲-۴
۲۶	 مثال‌های کاربردی	۳-۲-۴
۲۷	 مزایا و معایب	۴-۲-۴
۲۸	 مدل جایگزین محلی (Local Surrogate / LIME)	۳-۴
۲۹	 LIME برای داده‌های جدولی	۱-۳-۴
۲۹	 LIME برای متن و تصویر	۲-۳-۴
۳۰	 پیاده‌سازی LIME	۳-۳-۴
۳۲	 مزایا و معایب	۴-۳-۴
۳۳	 مقادیر شاپلی (Shapley Values)	۴-۴
۳۳	 ایده کلی و منطق پایه	۱-۴-۴
۳۳	 ریاضیات و تئوری مقدار شاپلی	۲-۴-۴
۳۴	 تخمین مقادیر شاپلی	۳-۴-۴
۳۴	 پیاده‌سازی SHAP	۴-۴-۴
۳۷	 مزایا و معایب	۵-۴-۴

۳۷		اهمیت ویژگی (Feature Importance)	۵-۴
۳۷		مبانی نظری و الگوریتم	۱-۵-۴
۳۸		بررسی مجموعه داده مناسب برای محاسبه اهمیت ویژگی	۲-۵-۴
۳۹		مزایا و معایب	۳-۵-۴
۴۰		توضیحات مبتنی بر نمونه	۵
۴۰		نمونه‌های اولیه و استثنایا (Prototypes and Criticisms)	۱-۵
۴۰		تئوری MMD-critic	۱-۱-۵
۴۱		یافتن استثنایا و تابع شاهد	۲-۱-۵
۴۱		کاربردهای MMD-critic در یادگیری ماشین تفسیرپذیر	۳-۱-۵
۴۲		توضیحات خلاف واقع (Counterfactual Explanations)	۲-۵
۴۲		معیارهای یک توضیح خلاف واقع مطلوب	۱-۲-۵
۴۳		تولید توضیحات خلاف واقع	۲-۲-۵
۴۴		مثال‌های کاربردی	۳-۲-۵
۴۴		مزایا و معایب	۴-۲-۵
۴۵		نمونه‌های متخاصم (Adversarial Examples)	۳-۵
۴۵		روش‌ها و مثال‌های تولید نمونه‌های متخاصم	۱-۳-۵

۱-۱ یادگیری ماشین چیست؟

یادگیری ماشین مجموعه‌ای از روش‌هاست که کامپیوترها برای انجام و بهبود پیش‌بینی‌ها یا رفتارها بر اساس داده‌ها از آن‌ها استفاده می‌کنند. به عنوان مثال، برای پیش‌بینی قیمت یک خانه، کامپیوتر الگوها را از فروش‌های قبلی خانه یاد می‌گیرد. تمرکز این گزارش بر یادگیری ماشین نظارت‌شده است که در آن‌ها مجموعه‌ای از داده‌ها را در اختیار داریم که خروجی مورد نظر (مثلاً قیمت‌های قبلی خانه) را از قبل در آن‌ها می‌دانیم و می‌خواهیم پیش‌بینی این خروجی را برای داده‌های جدید یاد بگیریم.

از یادگیری نظارت‌شده وظایفی مانند خوشه‌بندی که در دسته یادگیری غیرنظارت‌شده قرار می‌گیرند مستثنی هستند، چرا که در آن‌ها خروجی مشخصی نداریم و فقط می‌خواهیم دسته‌ها یا خوشه‌هایی از نقاط داده را پیدا کنیم. همچنین مواردی مانند یادگیری تقویتی نیز مستثنی هستند که در آن یک عامل یاد می‌گیرد تا با اقدام و پاسخ در یک محیط، پاداش مشخصی را به حداکثر برساند (مانند کامپیوتری که بازی تتریس انجام می‌دهد).

هدف یادگیری نظارت‌شده، یادگیری یک مدل پیش‌بینی‌کننده است که ویژگی‌های داده‌ها مانند متر از خانه، موقعیت مکانی، نوع کف‌پوش و غیره را به یک خروجی مانند قیمت خانه نگاشت کند. اگر خروجی از نوع دسته‌بندی باشد، به این کار طبقه‌بندی می‌گویند و اگر خروجی عددی باشد، به آن رگرسیون گفته می‌شود. الگوریتم یادگیری ماشین، یک مدل را از طریق تخمین پارامترها مانند وزن‌ها یا یادگیری ساختارها مانند درخت‌ها یاد می‌گیرد. این الگوریتم توسط یک تابع امتیاز یا تابع زیان که هدفش به حداقل رساندن خطا است، هدایت می‌شود. در مثال ارزش خانه، ماشین تفاوت بین قیمت تخمینی و قیمت واقعی پیش‌بینی‌شده را به حداقل می‌رساند. یک مدل یادگیری ماشین که به‌طور کامل آموزش دیده است، می‌تواند برای پیش‌بینی نمونه‌های جدید مورد استفاده قرار گیرد.

۱-۲ فرآیند ۳ مرحله‌ای یادگیری ماشین

تخمین قیمت خانه، توصیه‌گرهای محصول، تشخیص علائم راهنمایی و رانندگی، عدم پرداخت وام و تشخیص کلاهبرداری، همه این مثال‌ها در یک چیز مشترک هستند و آن این است که می‌توان آن‌ها را با یادگیری ماشین حل کرد. با وجود اینکه وظایف متفاوت هستند، اما رویکرد نهایی یکسان است.

در مرحله اول که جمع‌آوری داده‌هاست، هر چه داده بیشتری داشته باشیم بهتر است؛ این داده‌ها باید شامل خروجی مورد نظر برای پیش‌بینی و اطلاعات اضافی باشند که پیش‌بینی بر اساس آن‌ها انجام می‌شود. به عنوان مثال برای یک آشکارساز تابلوی راهنمایی و رانندگی که می‌خواهد وجود تابلو را در تصویر بررسی کند، تصاویر خیابان جمع‌آوری شده و برچسب می‌خورند. برای پیش‌بینی میزان اعتبار نیز به داده‌های گذشته در مورد وام‌ها، اطلاعات قصور مشتریان در پرداخت و داده‌های کمکی مثل درآمد نیاز است. همچنین برای تخمین ارزش خانه، داده‌های فروش گذشته و مشخصات ملک جمع‌آوری می‌شود.

در مرحله دوم، این اطلاعات وارد یک الگوریتم یادگیری ماشین می‌شود تا مدل نهایی (مانند مدل رتبه‌بندی اعتبار یا تخمین گر ارزش خانه) تولید شود.

در مرحله سوم و نهایی، از این مدل با داده‌های جدید استفاده می‌شود و آن را در یک محصول یا فرآیند واقعی مانند خودروی خودران یا وب‌سایت بازار مسکن ادغام می‌کنند.

۳-۱ مزایا و معایب یادگیری ماشین

ماشین‌ها در بسیاری از وظایف از انسان‌ها پیشی می‌گیرند، مانند بازی شطرنج، بازی گویا پیش‌بینی آب و هوا. حتی اگر ماشین در یک کار به اندازه انسان یا کمی ضعیف‌تر عمل کند، همچنان مزایای بزرگی از نظر سرعت، تکرارپذیری و مقیاس‌پذیری دارد.

یک مدل یادگیری ماشین که یک‌بار پیاده‌سازی شده است، می‌تواند کاری را بسیار سریع‌تر از انسان‌ها انجام دهد، نتایج ثابتی را به طور قابل‌اعتمادی ارائه دهد و به دفعات بی‌نهایت و با هزینه‌ای ناچیز کپی شود، در حالی که آموزش یک انسان برای یک کار می‌تواند ده‌ها طول بکشد و بسیار هزینه‌بر باشد.

با این حال، بزرگترین عیب استفاده از یادگیری ماشین این است که بینش درباره داده‌ها و مسئله‌ای که ماشین حل می‌کند، در مدل‌هایی که روز به روز پیچیده‌تر می‌شوند، پنهان می‌ماند. شما به میلیون‌ها عدد برای توصیف یک شبکه عصبی عمیق نیاز دارید و هیچ راهی برای درک کل مدل به صورت یکپارچه وجود ندارد.

مدل‌های دیگر، مانند جنگل تصادفی، از صدها درخت تصمیم تشکیل شده‌اند که برای پیش‌بینی‌ها رأی می‌دهند و برای درک نحوه تصمیم‌گیری، باید به آرا و ساختار هر یک از صدها درخت نگاه کنید که این کار عملاً فراتر از ظرفیت حافظه کاری انسان است. بهترین مدل‌ها از نظر عملکرد، اغلب ترکیبی از چندین مدل هستند که به آن‌ها مدل‌های ترکیبی^۱ گفته می‌شود و حتی اگر هر مدل به تنهایی قابل تفسیر باشد، کل مجموعه قابل تفسیر نخواهد بود. اگر فقط روی عملکرد تمرکز کنید، به طور خودکار مدل‌های مبهم‌تر و پیچیده‌تری خواهید داشت؛ همانطور که در پلتفرم مسابقات کگل مشاهده می‌شود، مدل‌های برنده اکثراً ترکیبی از مدل‌ها یا ساختارهای بسیار پیچیده مانند درخت‌های تقویت‌شده یا شبکه‌های عصبی عمیق هستند.

۴-۱ اصطلاحات تخصصی

برای جلوگیری از سردرگمی ناشی از ابهام، تعاریف دقیقی برای اصطلاحات این کتاب ارائه شده است.

الگوریتم^۲ مجموعه‌ای از قوانین است که یک ماشین برای دستیابی به یک هدف خاص دنبال می‌کند و می‌توان آن را مانند یک دستور پخت غذا دانست که ورودی‌ها، خروجی و تمام مراحل لازم را تعریف می‌کند.

یادگیری ماشین^۳ مجموعه‌ای از روش‌ها است که به کامپیوترها اجازه می‌دهد تا از داده‌ها برای ایجاد و بهبود پیش‌بینی‌ها یاد بگیرند و این یک تغییر پارادایم از برنامه‌نویسی معمولی (که تمام دستورالعمل‌ها صریحاً داده می‌شوند) به برنامه‌نویسی غیرمستقیم از طریق ارائه داده است.

الگوریتم یادگیری ماشین، همان برنامه‌ای است که برای یادگیری یک مدل از روی داده‌ها استفاده می‌شود.

مدل یادگیری ماشین، برنامه یادگرفته‌شده‌ای است که ورودی‌ها را به پیش‌بینی‌ها نگاشت می‌کند؛ این مدل می‌تواند مجموعه‌ای از وزن‌ها برای یک مدل خطی یا شبکه عصبی باشد و با نام‌های دیگری چون پیش‌بینی‌کننده، طبقه‌بندی‌کننده یا مدل رگرسیون نیز شناخته می‌شود و در فرمول‌ها با $\hat{f}$ یا $\hat{f}(x)$ نشان داده می‌شود.

¹Ensemble

²Algorithm

³Machine Learning

مدل جعبه سیاه^۴ سیستمی است که مکانیسم‌های داخلی خود را آشکار نمی‌کند و با نگاه کردن به پارامترهایش قابل درک نیست؛ نقطه مقابل آن جعبه سفید یا مدل تفسیرپذیر است و روش‌های مستقل از مدل برای تفسیرپذیری، با مدل‌ها مانند جعبه سیاه رفتار می‌کنند حتی اگر اینطور نباشند.

یادگیری ماشین تفسیرپذیر^۵ به روش‌ها و مدل‌هایی اشاره دارد که رفتار و پیش‌بینی‌های سیستم‌های یادگیری ماشین را برای انسان‌ها قابل درک می‌کنند.

مجموعه داده^۶ جدولی حاوی داده‌هایی است که ماشین از آن‌ها یاد می‌گیرد و شامل ویژگی‌ها و هدف است که هنگام آموزش به آن داده‌های آموزشی می‌گویند. نمونه به یک سطر در مجموعه داده گفته می‌شود که نام‌های دیگری چون نقطه داده، مثال یا مشاهده دارد و شامل مقادیر ویژگی^۷ $x^{(i)}$ و خروجی هدف y_i است.

ویژگی‌ها^۷ ورودی‌هایی در قالب ستون‌های مجموعه داده هستند که برای پیش‌بینی یا طبقه‌بندی استفاده می‌شوند و فرض بر این است که تفسیرپذیر هستند، چرا که اگر درک ویژگی‌های ورودی سخت باشد، درک عملکرد مدل حتی سخت‌تر خواهد بود؛ ماتریس تمام ویژگی‌ها با X ، ویژگی یک نمونه با $x^{(i)}$ و مقدار ویژگی j برای نمونه i با $x_j^{(i)}$ نشان داده می‌شود.

هدف^۸ همان اطلاعاتی است که ماشین یاد می‌گیرد تا آن را پیش‌بینی کند و در فرمول‌ها معمولاً y یا y_i نامیده می‌شود. وظیفه یادگیری ماشین ترکیب یک مجموعه داده همراه با ویژگی‌ها و یک هدف است که می‌تواند طبقه‌بندی، رگرسیون، تحلیل بقا، خوشه‌بندی یا تشخیص داده‌های پرت باشد. در نهایت، پیش‌بینی همان حدس مدل یادگیری ماشین برای مقدار هدف بر اساس ویژگی‌های داده‌شده است که در این کتاب با $\hat{f}(x^{(i)})$ یا $\hat{y}$ نشان داده می‌شود.

⁴Black-box model

⁵Interpretable Machine Learning

⁶Dataset

⁷Features

⁸Target

۲ تفسیرپذیری

هیچ تعریف ریاضی دقیقی برای تفسیرپذیری وجود ندارد. یک تعریف (غیر ریاضی) از میلر [۱] است: تفسیرپذیری میزانی است که یک انسان می‌تواند علت یک تصمیم را درک کند.

تعریف دیگر این است: تفسیرپذیری میزانی است که یک انسان می‌تواند نتیجه مدل را به طور مداوم و سازگار پیش‌بینی کند. هرچه تفسیرپذیری یک مدل یادگیری ماشین بالاتر باشد، درک اینکه چرا تصمیمات یا پیش‌بینی‌های خاصی انجام شده است برای افراد آسان‌تر خواهد بود. یک مدل زمانی بهتر از مدل دیگر تفسیرپذیر است که درک تصمیمات آن برای انسان نسبت به مدل دیگر آسان‌تر باشد.

۱-۲ اهمیت تفسیرپذیری

اگر یک مدل یادگیری ماشین عملکرد خوبی دارد، چرا فقط به آن اعتماد نکنیم و اینکه چرا چنین تصمیمی گرفته است را نادیده نگیریم؟

دوشی-ولز و همکاران [۲] بیان می‌کنند که مشکل این است که یک معیار واحد، مانند دقت دسته‌بندی، توصیف ناقصی از اکثر وظایف در دنیای واقعی است. وقتی صحبت از مدل‌سازی پیش‌بینانه می‌شود، شما باید یک مصالحه^۹ انجام دهید. آیا فقط می‌خواهید بدانید چه چیزی پیش‌بینی شده است؟ یا اینکه می‌خواهید بدانید چرا پیش‌بینی انجام شده و احتمالاً هزینه این تفسیرپذیری را با کاهش عملکرد پیش‌بینی بپردازید؟ در برخی موارد، برای شما مهم نیست که چرا تصمیمی گرفته شده است؛ اما در موارد دیگر، دانستن چرایی می‌تواند به شما کمک کند تا درباره مسئله، داده‌ها و دلیل شکست احتمالی مدل بیشتر بیاموزید.

دلایل متعددی تقاضا برای تفسیرپذیری و توضیحات را هدایت می‌کنند. از منظر کنجکاوی و یادگیری، انسان‌ها مدل ذهنی مختص به خود را از محیط دارند که با وقوع اتفاقات غیرمنتظره و یافتن توضیحی برای آن‌ها به‌روز می‌شود. همچنین در راستای یافتن معنا در جهان، ما همواره می‌خواهیم تضادهای یا ناسازگاری‌های بین ساختارهای دانش خود را هماهنگ کنیم؛ بر این اساس، هرچه تصمیم یک ماشین بیشتر بر زندگی یک فرد تأثیر بگذارد، اهمیت توضیح رفتار آن ماشین نیز بیشتر می‌شود. در بسیاری از رشته‌های علمی که با کلان‌داده‌ها سروکار دارند، مدل به جای داده‌ها به منبع دانش تبدیل می‌شود و تفسیرپذیری استخراج این دانش نهفته را ممکن می‌سازد.

علاوه بر این، مدل‌های یادگیری ماشین امروزه وظایفی را بر عهده می‌گیرند که نیازمند اقدامات ایمنی و تست هستند (نظیر خودروهای خودران) تا از بروز فجایع جلوگیری شود. از آنجا که این مدل‌ها ممکن است سوگیری‌هایی^{۱۰} را از داده‌های آموزشی بیاموزند، تفسیرپذیری به عنوان یک ابزار خطایابی کارآمد برای تشخیص تبعیض عمل می‌کند. در نهایت، فرآیند ادغام ماشین‌ها در زندگی روزمره ما، برای افزایش پذیرش اجتماعی و همچنین تسهیل فرآیند دیباگ و حسابرسی، نیازمند شفافیت است. اگر اطمینان حاصل کنید که مدل می‌تواند تصمیمات خود را توضیح دهد، بررسی ویژگی‌هایی مانند انصاف^{۱۱}، حفظ حریم خصوصی^{۱۲}، قابلیت اطمینان یا استواری^{۱۳}، کشف روابط علی^{۱۴} و در نهایت جلب اعتماد^{۱۵} کاربران

⁹Trade-off

¹⁰Bias

¹¹Fairness

¹²Privacy

¹³Robustness

¹⁴Causality

¹⁵Trust

بسیار آسان‌تر خواهد شد.

۲-۲ چه زمانی به تفسیرپذیری نیاز نداریم؟

با وجود اهمیت فراوان تفسیرپذیری، در سناریوهایی خاص نیازی به آن نداریم. به عنوان مثال، زمانی که مدل تأثیر قابل توجهی بر ابعاد مالی یا اجتماعی انسان‌ها ندارد و خطای آن صرفاً برای سازنده کمی خجالت‌آور است، نیازی به درک چرایی خروجی مدل احساس نمی‌شود. همچنین در کاربردهایی که پیش‌تر به اندازه کافی مطالعه شده‌اند و سال‌ها تجربه عملی موفق پشت آن‌هاست (نظیر سیستم‌های تشخیص نوری نویسه‌ها)، ضرورتی برای استخراج بینش‌های اضافی وجود ندارد.

افزون بر این، در سیستم‌هایی که در آن‌ها اهداف سازندگان با منافع کاربران در تضاد است (مانند سیستم‌های امتیازدهی اعتباری بانک‌ها)، تفسیرپذیری بالا می‌تواند نقطه ضعف محسوب شود؛ چرا که به افراد آگاه امکان می‌دهد تا با فریب دادن و دستکاری عمدی ورودی‌ها، سیستم را دور بزنند.

۳-۲ طبقه‌بندی روش‌های تفسیرپذیری

روش‌های تفسیرپذیری یادگیری ماشین را می‌توان بر اساس معیارهای مختلفی طبقه‌بندی کرد.

نخستین معیار، تفاوت میان روش‌های ذاتی و پسینی^{۱۶} است؛ به این معنا که آیا تفسیرپذیری با محدود کردن پیچیدگی خود مدل (مانند یک درخت تصمیم کوتاه) به دست می‌آید یا با اعمال روش‌هایی برای تحلیل یک مدل از پیش آموزش دیده.

معیار دوم به خروجی روش تفسیر اشاره دارد که می‌تواند شامل ارائه آمار خلاصه برای هر ویژگی، تجسم بصری این خلاصه‌ها، نمایش اجزای داخلی مدل نظیر وزن‌ها، تولید و معرفی نقاط داده جدید برای توجه پیش‌بینی، و در نهایت تقریب زدن با یک مدل کاملاً تفسیرپذیر باشد.

معیار سوم بررسی می‌کند که آیا روش مورد نظر منحصراً برای یک کلاس مدل خاص طراحی شده است یا به صورت مستقل از مدل^{۱۷} عمل می‌کند.

معیار چهارم و نهایی نیز محلی یا سراسری بودن^{۱۸} تفسیرپذیری است؛ یعنی مشخص می‌کند که آیا روش ارائه‌شده تنها یک پیش‌بینی منفرد را توضیح می‌دهد یا رفتار کلی مدل را در بر می‌گیرد.

۴-۲ محدوده تفسیرپذیری

فرآیند تفسیرپذیری در سطوح و محدوده‌های مختلفی قابل تحلیل است. در عام‌ترین سطح، شفافیت الگوریتم مطرح است که بررسی می‌کند الگوریتم چگونه مدل را از روی داده‌ها یاد می‌گیرد، بی‌آنکه وارد جزئیات پیش‌بینی نهایی شود.

سطح بعدی، تفسیرپذیری سراسری و جامع مدل^{۱۹} است که تلاش می‌کند نحوه تصمیم‌گیری مدل آموزش دیده را به طور یکپارچه توضیح دهد؛ هدفی که در مدل‌های پیچیده عملاً فراتر از توان انسان است.

¹⁶Intrinsic or post hoc

¹⁷Model-specific or model-agnostic

¹⁸Local or global

¹⁹Global, Holistic Model Interpretability

سطح واقع‌بینانه‌تر، تفسیرپذیری سراسری در سطح ماژولار^{۲۰} است که نشان می‌دهد بخش‌ها و اجزای خاصی از مدل (نظیر تک‌تک وزن‌ها در مدل خطی) چگونه بر پیش‌بینی‌ها تأثیر می‌گذارند.

در خردترین سطوح، تفسیرپذیری محلی برای یک پیش‌بینی واحد^{۲۱} به ما اجازه می‌دهد روی یک نمونه خاص متمرکز شده و چرایی پیش‌بینی آن را بررسی کنیم. با بسط دادن همین مفهوم می‌توان به تفسیرپذیری محلی برای گروهی از پیش‌بینی‌ها دست یافت که منطق تصمیم‌گیری مدل را برای زیرمجموعه خاصی از داده‌ها روشن می‌سازد.

۲-۵ ارزیابی تفسیرپذیری

گرچه توافق قطعی در مورد چستی تفسیرپذیری و نحوه دقیق سنجش آن وجود ندارد، اما دوشی-ولز و کیم^[۲] سه سطح اصلی را برای ارزیابی آن پیشنهاد داده‌اند.

سطح اول، ارزیابی در سطح کاربرد^{۲۲} است که در آن، توضیح تولید شده در دل یک نرم‌افزار کاربردی گنجانده شده و مستقیماً توسط متخصصان آن حوزه (مانند رادیولوژیست‌ها) ارزیابی می‌شود.

سطح دوم، ارزیابی در سطح انسان^{۲۳} نام دارد که نسخه‌ای ساده‌شده از رویکرد پیشین است؛ با این تفاوت که وظایف به جای متخصصان، توسط افراد عادی سنجیده می‌شوند تا هزینه و زمان آزمایش کاهش یابد.

سطح سوم، ارزیابی در سطح عملکرد^{۲۴} است که مستقل از آزمایش انسانی انجام می‌شود و در آن از معیارهای جانشین (پروکسی) مانند عمق یک درخت تصمیم، به عنوان شاخصی برای سنجش کیفیت و سهولت فهم توضیحات استفاده می‌گردد.

۲-۶ ویژگی‌های توضیحات

برای ارزیابی و سنجش میزان کیفیت یک روش توضیح و همچنین خروجی آن، از مجموعه‌ای از ویژگی‌های بنیادین استفاده می‌شود.

در مورد ویژگی‌های روش‌های توضیح، مواردی چون قدرت بیان که نوع ساختار توضیحات (قواعد شرطی، درخت تصمیم و غیره) را تعیین می‌کند، و شفافیت درونی که وابستگی روش به بررسی درون جعبه سیاه مدل را می‌سنجد، بسیار حائز اهمیت‌اند.

همچنین میزان قابلیت جابجایی روی مدل‌های مختلف و پیچیدگی الگوریتمی برای تولید این توضیحات نیز از دیگر فاکتورهای کلیدی محسوب می‌شوند.

در سمت دیگر، خود توضیحات فردی تولید شده نیز باید ویژگی‌های مشخصی داشته باشند. دقت^{۲۵} نشان می‌دهد که یک توضیح تا چه حد برای پیش‌بینی روی نمونه‌های جدید کارایی دارد، و وفاداری^{۲۶} مشخص می‌کند که توضیح تا چه اندازه توانسته رفتار واقعی مدل جعبه سیاه را تقریب بزند.

²⁰Global Model Interpretability on a Modular Level

²¹Local Interpretability for a Single Prediction

²²Application level evaluation

²³Human level evaluation

²⁴Function level evaluation

²⁵Accuracy

²⁶Fidelity

همچنین سازگاری، ثبات توضیحات را در بین مدل‌های مشابه با نتایج یکسان می‌سجد، در حالی که پایداری، تضمین می‌کند که اعمال تغییرات جزئی در ورودی‌ها منجر به تغییرات شدید در توضیحات نشود.

در نهایت قابلیت درک توسط انسان، قطعیت مدل، بیان صحیح درجه اهمیت ویژگی‌ها و میزان پوشش یا نمایندگی یک توضیح برای سایر نمونه‌ها، از ارکان اساسی برای داشتن یک یادگیری ماشین تفسیرپذیر معتبر به شمار می‌روند.

۳ مدل های تفسیرپذیر

ساده ترین راه برای دستیابی به تفسیرپذیری، استفاده از زیرمجموعه ای از الگوریتم هاست که مدل های تفسیرپذیر^{۲۷} ایجاد می کنند. رگرسیون خطی^{۲۸}، رگرسیون لجستیک^{۲۹} و درختان تصمیم^{۳۰} از جمله مدل های تفسیرپذیر بسیار رایج هستند.

در این بخش، به بررسی مفاهیم پایه این مدل ها و نحوه تفسیر آن ها می پردازیم. تمامی مدل های بررسی شده در این بخش، به استثنای روش k همسایه نزدیک^{۳۱}، در سطح ماژولار قابل تفسیر هستند.

یک مدل زمانی خطی است که ارتباط بین ویژگی ها^{۳۲} و هدف^{۳۳} به صورت خطی مدل سازی شود. مدلی با محدودیت های یکنوایی^{۳۴} تضمین می کند که رابطه بین یک ویژگی و خروجی هدف همواره در یک جهت مشخص حرکت کند که این امر تفسیر مدل را ساده تر می کند. همچنین، برخی مدل ها می توانند به طور خودکار تعاملات بین ویژگی ها^{۳۵} را در نظر بگیرند، هرچند تعاملات بیش از حد پیچیده می تواند به تفسیرپذیری آسیب برساند.

۱-۳ رگرسیون خطی (Linear Regression)

یک مدل رگرسیون خطی، هدف را به عنوان مجموع وزن دار ویژگی های ورودی پیش بینی می کند. خطی بودن این رابطه یادگرفته شده، تفسیر آن را بسیار ساده می سازد. روابط یادگرفته شده در این مدل ها خطی هستند و برای یک نمونه منفرد i به صورت زیر نوشته می شوند:

$$y = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p + \epsilon$$

در این معادله، خروجی پیش بینی شده برای یک نمونه، مجموع وزن دار p ویژگی آن است. بتاها (β_j) نشان دهنده وزن ها یا ضرایب یادگرفته شده ویژگی ها هستند. اولین وزن در این مجموع (β_0) عرض از مبدأ^{۳۶} نامیده می شود و در هیچ ویژگی ضرب نمی گردد. اپسیلون (ϵ) خطای مدل است؛ یعنی تفاوت بین پیش بینی و خروجی واقعی. فرض بر این است که این خطاها از یک توزیع گاوسی^{۳۷} پیروی می کنند.

برای تخمین وزن های بهینه معمولاً از روش حداقل مربعات معمولی^{۳۸} استفاده می شود تا مجموع مربعات تفاضل بین خروجی های واقعی و پیش بینی شده حداقل گردد:

$$\hat{\beta} = \arg \min_{\beta_0, \dots, \beta_p} \sum_{i=1}^n \left(y^{(i)} - \left(\beta_0 + \sum_{j=1}^p \beta_j x_j^{(i)} \right) \right)^2$$

وزن های تخمین زده شده همراه با بازه های اطمینان^{۳۹} ارائه می شوند؛ به عنوان مثال، یک بازه اطمینان ۹۵ درصدی به این

²⁷Interpretable Models

²⁸Linear Regression

²⁹Logistic Regression

³⁰Decision Trees

³¹k-nearest neighbors

³²Features

³³Target

³⁴Monotonicity Constraints

³⁵Feature Interactions

³⁶Intercept

³⁷Gaussian Distribution

³⁸Ordinary Least Squares (OLS)

³⁹Confidence Intervals

معناست که اگر تخمین را ۱۰۰ بار با داده‌های نمونه‌گیری شده جدید تکرار کنیم، در ۹۵ مورد وزن واقعی در این بازه قرار می‌گیرد.

۳-۱-۱ مفروضات مدل رگرسیون خطی

اعتبار این مدل و تفسیرپذیری آن به رعایت مجموعه‌ای از مفروضات آماری بستگی دارد. نخستین فرض، خطی بودن است که پیش‌بینی را مجبور می‌کند ترکیب خطی از ویژگی‌ها باشد؛ این ویژگی‌ها اثرات خطی افزایشی دارند و به راحتی قابل تفکیک و کمی‌سازی هستند. فرض دوم، نرمال بودن است که بیان می‌کند خروجی هدف با توجه به ویژگی‌ها دارای توزیع نرمال است و در صورت نقض آن، بازه‌های اطمینان نامعتبر خواهند بود. علاوه بر این، فرض هم‌واریانسی ایجاب می‌کند که واریانس عبارات خطا در کل فضای ویژگی ثابت باشد. استقلال نمونه‌ها از یکدیگر، در نظر گرفتن ویژگی‌های ورودی به عنوان ثوابت بدون خطای اندازه‌گیری و همچنین عدم وجود هم‌خطی چندگانه از دیگر مفروضات اساسی هستند. وجود همبستگی شدید بین ویژگی‌ها، تخمین وزن‌ها را مختل کرده و تشخیص اثر هر ویژگی را غیرممکن می‌سازد.

۳-۱-۲ تفسیر ضرایب و معیار ارزیابی

تفسیر یک وزن در مدل رگرسیون خطی کاملاً به نوع ویژگی متناظر با آن بستگی دارد. برای یک ویژگی عددی، افزایش یک واحد در ویژگی، خروجی پیش‌بینی شده را دقیقاً به اندازه وزن آن ویژگی تغییر می‌دهد. در مورد ویژگی‌های دودوای و طبقه‌ای که معمولاً از رمزگذاری وان-هات^{۴۰} برای آن‌ها استفاده می‌شود، تغییر ویژگی از دسته مرجع به دسته دیگر باعث تغییر خروجی به اندازه وزن آن ویژگی می‌گردد. همچنین عرض از مبدأ (β_0) نشان‌دهنده پیش‌بینی مدل برای نمونه‌ای است که تمام ویژگی‌های عددی آن صفر و ویژگی‌های طبقه‌ای آن در دسته مرجع قرار داشته باشند. برای ارزیابی میزان توضیح‌دهندگی مدل، از معیار R^2 استفاده می‌شود که نشان می‌دهد چه مقدار از واریانس کل خروجی هدف توسط مدل توضیح داده شده است:

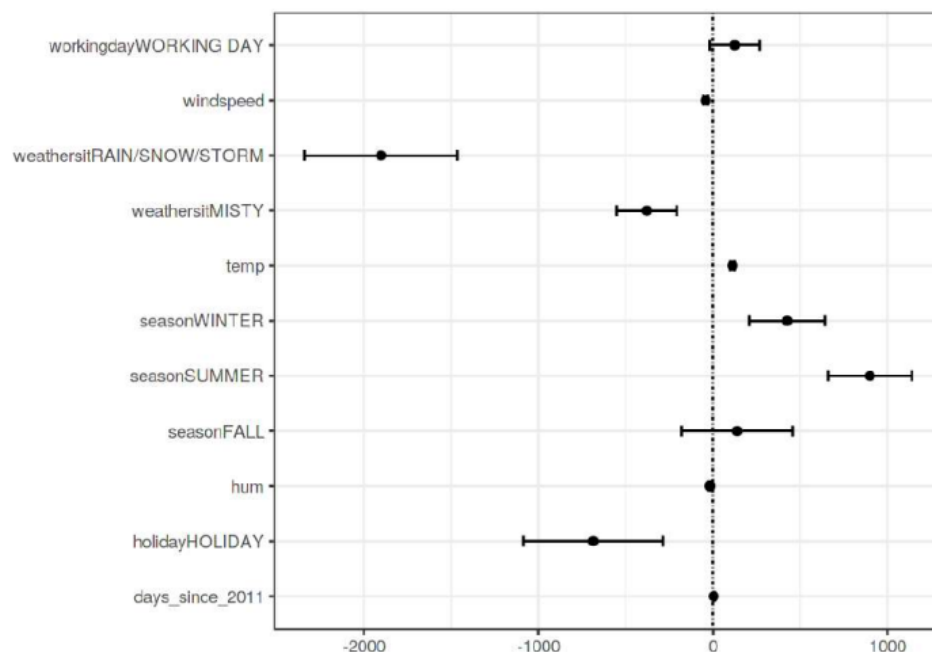
$$R^2 = 1 - SSE/SST$$

که در آن SSE مجموع مربعات خطاها و SST مجموع مربعات واریانس داده‌ها است. از آنجا که با افزودن ویژگی‌های جدید (حتی بی‌ربط) مقدار R^2 افزایش می‌یابد، برای دستیابی به ارزیابی دقیق‌تر در متون علمی، استفاده از R^2 تعدیل‌شده^{۴۱} که تعداد ویژگی‌ها را نیز لحاظ می‌کند، توصیه می‌شود:

$$\bar{R}^2 = R^2 - (1 - R^2) \frac{p}{n - p - 1}$$

^{۴۰}One-hot encoding

^{۴۱}Adjusted R-squared



شکل ۱: نمودار وزن‌ها در رگرسیون لجستیک. وزن‌ها به صورت نقاط نمایش داده شده‌اند و خطوط، بازه‌های اطمینان ۹۵٪ متناظر را نشان می‌دهند.

۳-۱-۳ اهمیت ویژگی‌ها و تفسیر بصری

اهمیت یک ویژگی در مدل رگرسیون خطی با مقدار قدر مطلق آماره t ^{۴۲} اندازه‌گیری می‌شود که برابر است با وزن تخمین زده شده تقسیم بر خطای استاندارد آن:

$$t_{\hat{\beta}_j} = \frac{\hat{\beta}_j}{SE(\hat{\beta}_j)}$$

برای تحلیل راحت‌تر این مدل‌ها، استفاده از ابزارهای بصری بسیار کارآمد است. نمودار وزن^{۴۳} وزن‌ها را به عنوان نقطه و بازه اطمینان آن‌ها را به عنوان خطوط نمایش می‌دهد. همچنین نمودار اثر^{۴۴} توزیع اثرات (ضرب وزن هر ویژگی در مقدار واقعی آن در داده‌ها) را با استفاده از نمودارهای جعبه‌ای^{۴۵} در کل مجموعه داده به تصویر می‌کشد.

۲-۳ رگرسیون لجستیک (Logistic Regression)

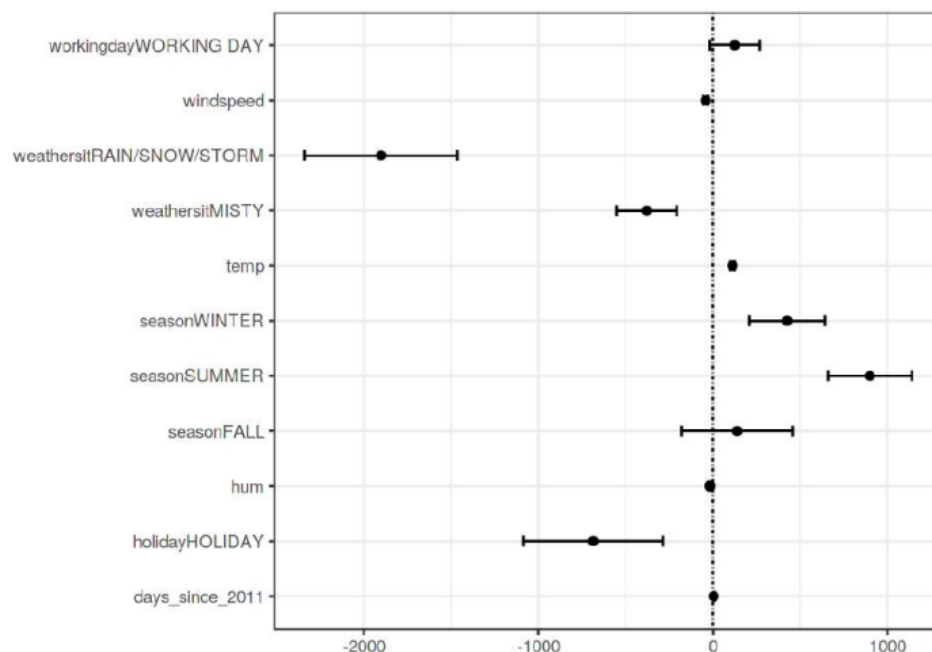
رگرسیون لجستیک احتمالات مربوط به مسائل طبقه‌بندی با دو خروجی ممکن را مدل‌سازی می‌کند. این روش در واقع توسعه و تعمیمی از مدل رگرسیون خطی برای وظایف طبقه‌بندی است.

⁴²t-statistic

⁴³Weight Plot

⁴⁴Effect Plot

⁴⁵Boxplots



شکل ۲: نمودار اثر ویژگی در رگرسیون لاجستیک. توزیع اثرات (= مقدار ویژگی ضرب در وزن ویژگی) برای هر ویژگی در میان داده‌ها نمایش داده شده است.

۱-۲-۳ چرا رگرسیون خطی برای طبقه‌بندی مناسب نیست؟

در یک مسئله طبقه‌بندی دوکلاسه، می‌توان یکی از کلاس‌ها را با ۰ و دیگری را با ۱ برچسب‌گذاری کرد و یک مدل رگرسیون خطی را روی داده‌ها برازش داد. هرچند از نظر فنی این کار شدنی است و الگوریتم‌ها می‌توانند وزن‌ها را برای شما محاسبه کنند، اما این رویکرد با چالش‌های جدی همراه است.

نخست اینکه، مدل خطی خروجی‌ای از جنس احتمال تولید نمی‌کند؛ بلکه با کلاس‌ها صرفاً به عنوان مقادیر عددی (۰ و ۱) رفتار کرده و بهترین ابرصفحه^{۴۶} (که در صورت وجود تنها یک ویژگی، یک خط راست است) را به گونه‌ای برازش می‌دهد که فواصل (خط‌ها) بین نقاط و ابرصفحه کمینه شود. بنابراین، خروجی مدل صرفاً یک درونیابی^{۴۷} بین نقاط است و نمی‌توان آن را به عنوان «احتمال وقوع یک کلاس» تفسیر کرد.

علاوه بر این، مدل خطی ممکن است دست به برون‌یابی^{۴۸} بزند و مقادیری کمتر از صفر یا بزرگ‌تر از یک تولید کند. این موضوع به وضوح نشان‌دهنده نیاز به یک رویکرد هوشمندانه‌تر برای طبقه‌بندی است. همچنین، از آنجایی که خروجی پیش‌بینی شده احتمال نیست، هیچ آستانه^{۴۹} معناداری برای تمایز و جداسازی کلاس‌ها از یکدیگر وجود نخواهد داشت.

در نهایت، مدل‌های خطی برای مسائل طبقه‌بندی چندکلاسه به درستی تعمیم نمی‌یابند؛ زیرا مجبور خواهید بود کلاس‌ها را با اعداد متوالی (مانند ۲، ۳، ...) برچسب‌گذاری کنید. در این حالت، حتی اگر کلاس‌ها ذاتاً هیچ ترتیب معناداری نداشته

^{۴۶}Hyperplane

^{۴۷}Interpolation

^{۴۸}Extrapolation

^{۴۹}Threshold

باشند، مدل خطی یک ساختار ترتیبی نامتعارف و کاذب را بر رابطه میان ویژگی‌ها و پیش‌بینی‌ها تحمیل می‌کند؛ به طوری که افزایش مقدار یک ویژگی با وزن مثبت، مدل را صرفاً به سمت پیش‌بینی کلاسی با شماره بالاتر سوق می‌دهد که از نظر مفهومی اشتباه است.

۲-۲-۳ تئوری مدل

راه‌حل بهینه برای این مسئله، رگرسیون لجستیک است. در این مدل به جای برازش یک خط راست یا ابرصفحه، از تابع لجستیک^{۵۰} استفاده می‌شود تا خروجی یک معادله خطی بین ۰ و ۱ فشرده شود. تابع لجستیک به صورت زیر تعریف می‌شود:

$$\text{logistic}(\eta) = \frac{1}{1 + \exp(-\eta)}$$

در رگرسیون خطی رابطه بین هدف و ویژگی‌ها به صورت زیر مدل‌سازی می‌شد:

$$\hat{y}^{(i)} = \beta_0 + \beta_1 x_1^{(i)} + \dots + \beta_p x_p^{(i)}$$

برای استخراج احتمالات بین ۰ و ۱ در طبقه‌بندی، سمت راست معادله فوق را درون تابع لجستیک قرار می‌دهیم:

$$P(y^{(i)} = 1) = \frac{1}{1 + \exp\left(-\left(\beta_0 + \beta_1 x_1^{(i)} + \dots + \beta_p x_p^{(i)}\right)\right)}$$

۳-۲-۳ تفسیر ضرایب

از آنجا که خروجی رگرسیون لجستیک یک احتمال بین ۰ و ۱ است، وزن‌ها دیگر تأثیر خطی و مستقیمی روی احتمال ندارند. برای تفسیر پذیری بهتر ضرایب، معادله را به گونه‌ای بازنویسی می‌کنیم که عبارت خطی در یک سمت قرار گیرد:

$$\log\left(\frac{P(y=1)}{1-P(y=1)}\right) = \log\left(\frac{P(y=1)}{P(y=0)}\right) = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$$

عبارت داخل لگاریتم «شانس»^{۵۱} (احتمال وقوع رویداد تقسیم بر احتمال عدم وقوع آن) نام دارد و فرم لگاریتمی آن، لگاریتم شانس^{۵۲} نامیده می‌شود. این فرمول نشان می‌دهد که رگرسیون لجستیک یک مدل خطی برای لگاریتم شانس است. با اعمال تابع نمایی $\exp(\cdot)$ بر طرفین، مشخص می‌شود که تغییر یک واحدی در ویژگی x_j ، نسبت شانس^{۵۳} را به صورت ضربی با فاکتور $\exp(\beta_j)$ تغییر می‌دهد.

تفسیر ضرایب بر اساس نوع ویژگی‌ها به شرح زیر است:

- **ویژگی عددی:** با افزایش یک واحد در مقدار ویژگی x_j ، شانس تخمین زده شده با فاکتور $\exp(\beta_j)$ تغییر (ضرب) می‌شود.

⁵⁰Logistic function

⁵¹Odds

⁵²Log odds

⁵³Odds ratio

- ویژگی طبقه‌ای دودویی: یکی از دسته‌ها به عنوان دسته مرجع^{۵۴} (معمولاً کدگذاری شده با ۰) در نظر گرفته می‌شود. تغییر وضعیت ویژگی از دسته مرجع به دسته دیگر، شانس را با فاکتور $\exp(\beta_j)$ تغییر می‌دهد.
- ویژگی طبقه‌ای چندکلاسه: با استفاده از رمزگذاری وان-هات، هر دسته ستون مجزایی دریافت کرده و تفسیر هر یک از آن‌ها مشابه ویژگی‌های دودویی خواهد بود.
- عرض از مبدأ (β_0): زمانی که تمام ویژگی‌های عددی صفر و ویژگی‌های طبقه‌ای در دسته مرجع باشند، شانس تخمین زده شده برابر با $\exp(\beta_0)$ خواهد بود.

۳-۲-۴ مزایا و معایب

رگرسیون لجستیک علاوه بر ارائه‌ی برچسب کلاس، احتمال وقوع آن را نیز مشخص می‌کند و به راحتی تحت عنوان رگرسیون چندجمله‌ای^{۵۵} به مسائل چندکلاسه تعمیم می‌یابد.

با این حال، این مدل در قدرت بیان دارای محدودیت است (تعاملات میان ویژگی‌ها باید دستی اضافه شوند) و تفسیر ضرایب آن به دلیل ساختار ضربی، از رگرسیون خطی پیچیده‌تر است. همچنین این مدل می‌تواند دچار مشکل جداسازی کامل^{۵۶} شود؛ یعنی اگر ویژگی خاصی وجود داشته باشد که کلاس‌ها را به طور کامل جدا کند، وزن آن به سمت بی‌نهایت میل کرده و مدل همگرا نمی‌شود که این چالش با معرفی روش‌های جریمه‌دهی (مانند تنظیم‌گرهای L_1 یا L_2) برطرف می‌شود.

۳-۳ درخت تصمیم (Decision Tree)

مدل‌های رگرسیون خطی و رگرسیون لجستیک^{۵۷} در شرایطی که رابطه بین ویژگی‌ها^{۵۸} و خروجی غیرخطی است یا ویژگی‌ها با یکدیگر تعامل دارند، با شکست مواجه می‌شوند. در اینجا است که درخت تصمیم^{۵۹} اهمیت پیدا می‌کند. مدل‌های مبتنی بر درخت، داده‌ها را چندین بار بر اساس مقادیر آستانه مشخصی در ویژگی‌ها تقسیم می‌کنند. از طریق این تقسیم‌بندی، زیرمجموعه‌های مختلفی از مجموعه داده‌ها ایجاد می‌شود که هر نمونه به یکی از آن‌ها تعلق می‌گیرد.

زیرمجموعه‌های نهایی به عنوان گره‌های پایانی یا برگ^{۶۰} و زیرمجموعه‌های میانی به عنوان گره‌های داخلی یا انشعاب^{۶۱} شناخته می‌شوند. برای پیش‌بینی خروجی در هر گره برگ، از میانگین خروجی داده‌های آموزشی در آن گره استفاده می‌گردد.

از درختان می‌توان هم برای طبقه‌بندی و هم برای رگرسیون استفاده کرد. الگوریتم‌های متنوعی برای رشد درخت وجود دارند که در ساختار درخت، معیارهای یافتن انشعاب، زمان توقف و نحوه تخمین مدل‌های ساده درون گره‌های برگ با یکدیگر تفاوت دارند. الگوریتم درختان طبقه‌بندی و رگرسیون که به اختصار CART^{۶۲} نامیده می‌شود، احتمالاً محبوب‌ترین الگوریتم برای استنتاج درخت است، هرچند منطق تفسیر برای اکثر انواع دیگر درختان نیز مشابه است.

⁵⁴Reference category

⁵⁵Multinomial Regression

⁵⁶Complete separation

⁵⁷Logistic regression

⁵⁸Features

⁵⁹Decision Tree

⁶⁰Terminal or leaf nodes

⁶¹Internal or split nodes

⁶²Classification and Regression Trees

رابطه بین خروجی y و ویژگی‌های x در این مدل‌ها با معادله زیر توصیف می‌شود:

$$\hat{y} = \hat{f}(x) = \sum_{m=1}^M c_m I\{x \in R_m\}$$

در این رابطه، هر نمونه دقیقاً در یک گره برگ (زیرمجموعه R_m) قرار می‌گیرد. تابع $I\{x \in R_m\}$ یک تابع نشانگر^{۶۳} است که اگر x در زیرمجموعه R_m باشد مقدار ۱ و در غیر این صورت مقدار ۰ را برمی‌گرداند. اگر نمونه‌ای در گره برگ R_l قرار گیرد، خروجی پیش‌بینی شده برابر با $\hat{y} = c_l$ خواهد بود، که در آن c_l میانگین تمام نمونه‌های آموزشی در گره برگ R_l است. الگوریتم CART برای یافتن این زیرمجموعه‌ها، ویژگی‌ها را بررسی کرده و نقطه برشی را تعیین می‌کند که واریانس^{۶۴} متغیر y را برای وظایف رگرسیون، یا شاخص جینی^{۶۵} توزیع کلاس y را برای وظایف طبقه‌بندی به حداقل برساند. الگوریتم این جستجو و تقسیم را به صورت بازگشتی در هر دو گره جدید ادامه می‌دهد تا زمانی که یک معیار توقف (مانند حداقل تعداد نمونه‌های مورد نیاز در یک گره) برآورده شود.

۱-۳-۳ تفسیر و اهمیت ویژگی‌ها

تفسیر این مدل‌ها بسیار ساده است: از گره ریشه^{۶۶} شروع کرده و مسیر لبه‌ها را دنبال می‌کنید تا به یک گره برگ برسید که خروجی پیش‌بینی شده را نشان می‌دهد؛ تمامی این لبه‌ها در منطق تصمیم‌گیری با عملگر «و» (AND) به یکدیگر متصل شده‌اند.

برای محاسبه اهمیت کلی یک ویژگی، تمام انشعاب‌هایی که آن ویژگی در آن‌ها استفاده شده است بررسی می‌شوند تا مشخص گردد که استفاده از آن، چقدر واریانس یا شاخص جینی را نسبت به گره والد کاهش داده است. مجموع اهمیت تمام ویژگی‌ها به مقیاس ۱۰۰ در می‌آید، بنابراین هر مقدار نشان‌دهنده سهم آن ویژگی از اهمیت کل مدل است.

۲-۳-۳ تجزیه درخت

پیش‌بینی‌های فردی یک درخت تصمیم را می‌توان با تجزیه مسیر تصمیم‌گیری به یک جزء برای هر ویژگی توضیح داد. گره ریشه نقطه شروع است که به صورت پیش‌فرض میانگین خروجی داده‌های آموزشی را پیش‌بینی می‌کند.

با هر انشعاب، بسته به مسیر طی شده، مقداری به این میانگین اضافه یا از آن کم می‌شود. بنابراین، پیش‌بینی یک نمونه منفرد برابر است با میانگین خروجی هدف به علاوه مجموع تمامی مشارکت‌های مربوط به انشعاب‌هایی که در مسیر آن نمونه رخ می‌دهند:

$$\hat{f}(x) = \bar{y} + \sum_{d=1}^D \text{split.contrib}(d, x) = \bar{y} + \sum_{j=1}^p \text{feat.contrib}(j, x)$$

از آنجا که یک ویژگی ممکن است در چندین انشعاب استفاده شود یا اصلاً استفاده نشود، با جمع کردن مشارکت‌های هر یک از p ویژگی، می‌توان به تفسیری دقیق از میزان تاثیر هر ویژگی در یک پیش‌بینی خاص دست یافت.

⁶³Identity/Indicator function

⁶⁴Variance

⁶⁵Gini index

⁶⁶Root node

۳-۳-۳ مزایا و معایب درخت تصمیم

ساختار درختی برای استخراج تعاملات بین ویژگی‌ها بسیار ایده‌آل است و داده‌ها را به گروه‌های متمایزی تقسیم می‌کند که درک آن‌ها اغلب ساده‌تر از مفاهیم انتزاعی در رگرسیون خطی است.

این مدل دارای یک تجسم بصری طبیعی است و تفاسیری مبتنی بر سناریوهای شرطی و خلاف واقع^{۶۷} ارائه می‌دهد. چنانچه درخت کوتاه باشد، توضیحات تولید شده بسیار ساده، گزینشی و عمومی خواهند بود.

علاوه بر این، در درختان تصمیم بر خلاف مدل‌های خطی، نیازی به تبدیل ویژگی‌ها (مانند استفاده از لگاریتم) نیست و درخت با هر تبدیل یکنوای ویژگی‌ها به خوبی کار می‌کند.

در مقابل، درختان تصمیم در مواجهه با روابط خطی به شدت ضعیف عمل می‌کنند، زیرا هر رابطه خطی باید توسط مجموعه‌ای از انشعاب‌ها تقریب زده شود که به یک تابع پله‌ای غیرکارآمد می‌انجامد. این مسئله با فقدان همواری^{۶۸} همراه است؛ به این معنا که تغییرات بسیار جزئی در ویژگی ورودی می‌تواند به دلیل تغییر مسیر در انشعاب‌ها، تأثیر بزرگی بر خروجی پیش‌بینی شده داشته باشد.

همچنین این مدل‌ها اساساً ناپایدار هستند و تغییرات اندک در داده‌های آموزشی می‌تواند ساختار درخت را کاملاً دگرگون سازد. در نهایت، بزرگترین چالش تفسیرپذیری درختان تصمیم محدودیت عمق آن‌هاست؛ با افزایش عمق درخت، تعداد گره‌های پایانی به صورت نمایی رشد کرده و درک جامع قوانین تصمیم‌گیری آن به شدت دشوار و پیچیده می‌شود.

۴-۳ قوانین تصمیم (Decision Rules)

یک قانون تصمیم، یک گزاره ساده «اگر-آنگاه» (IF-THEN) است که از یک شرط^{۶۹} و یک پیش‌بینی تشکیل می‌شود. به عنوان مثال، در گزاره «اگر امروز باران می‌بارد و اگر ماه آوریل است، آنگاه فردا باران خواهد بارید»، بخش اول شرط و بخش دوم پیش‌بینی مدل را نشان می‌دهد.

قوانین تصمیم به عنوان تفسیرپذیرترین مدل‌های پیش‌بینی شناخته می‌شوند، زیرا ساختار «اگر-آنگاه» آن‌ها از نظر معنایی بسیار شبیه به زبان طبیعی و نحوه تفکر انسان است؛ البته این تفسیرپذیری منوط به آن است که ویژگی‌ها قابل درک باشند، طول شرط کوتاه باشد و تعداد قوانین نیز محدود بماند.

۳-۴-۱ ساختار و مفاهیم پایه

یک قانون تصمیم به صورت کلی دارای یک ساختار مشخص است که از دو بخش اصلی تشکیل می‌شود.

بخش نخست یا بخش شرط (IF-part)، حداقل شامل یک گزاره به شکل «ویژگی=مقدار» است و در صورت نیاز می‌توان گزاره‌های بیشتری را با استفاده از عملگر «و» (AND) به آن اضافه کرد.

بخش دوم یا بخش پیش‌بینی (THEN-part)، خروجی مدل را در صورت برقرار بودن تمامی شروط بخش اول مشخص می‌کند. کارایی یک قانون تصمیم معمولاً با دو معیار اصلی ارزیابی و خلاصه می‌شود: پشتیبانی یا پوشش (Support /)

⁶⁷Counterfactuals

⁶⁸Lack of smoothness

⁶⁹Condition / Antecedent

Coverage) و دقت یا اطمینان (Accuracy / Confidence). پشتیبانی نشان‌دهنده درصد نمونه‌هایی از کل داده‌هاست که شرط قانون برای آن‌ها صدق می‌کند، در حالی که دقت نشان می‌دهد در میان نمونه‌هایی که قانون برای آن‌ها اعمال شده است، چه درصدی به درستی پیش‌بینی شده‌اند. معمولاً در تنظیم این قوانین بین دو معیار یادشده یک بده‌بستان^{۷۰} وجود دارد؛ به عبارت دیگر، با افزودن ویژگی‌های بیشتر به شرط، می‌توان دقت مدل را افزایش داد، اما این امر معمولاً به کاهش پشتیبانی و پوشش داده‌ها منجر می‌گردد.

۳-۴-۲ استراتژی‌های ترکیب قوانین

زمانی که برای حل یک مسئله به چندین قانون نیاز است، پژوهشگران با دو چالش اساسی مواجه می‌شوند: نخست «تداخل قوانین» که در آن چند قانون با پیش‌بینی‌های متناقض برای یک نمونه واحد صدق می‌کنند، و دوم «عدم شمول» که در آن هیچ قانونی برای یک نمونه خاص برقرار نیست.

برای غلبه بر این چالش‌ها، دو استراتژی اصلی تحت عناوین لیست‌های تصمیم و مجموعه‌های تصمیم به کار گرفته می‌شوند.

در رویکرد لیست‌های تصمیم که ساختاری مرتب‌سازی‌شده دارد، قوانین به صورت متوالی بررسی شده و پیش‌بینی اولین قانونی که شرط آن برقرار باشد، به عنوان خروجی بازگردانده می‌شود.

در مقابل، رویکرد مجموعه‌های تصمیم فاقد چنین ترتیبی بوده و مشابه یک سیستم رأی‌گیری (مانند رأی اکثریت) عمل می‌کند؛ هرچند باید توجه داشت که همپوشانی قوانین در این روش ممکن است به کاهش تفسیرپذیری مدل منجر شود.

شایان ذکر است که هر دو روش برای جلوگیری از مشکل عدم شمول، از یک قانون پیش‌فرض بهره می‌برند؛ این قانون اطمینان حاصل می‌کند که اگر هیچ‌یک از قوانین دیگر صدق نکردند، یک پیش‌بینی پایه اعمال شود تا فضای ویژگی به طور کامل پوشش داده شود.

۳-۴-۳ الگوریتم‌های یادگیری قوانین

روش‌های مختلفی برای استخراج قوانین از داده‌ها توسعه یافته‌اند که در ادامه سه رویکرد اصلی و متمایز معرفی می‌شوند.

یادگیری از یک ویژگی منفرد (OneR)

الگوریتم OneR^{۷۱} بر خلاف نامش، بیش از یک قانون ایجاد می‌کند و در واقع به ازای هر مقدار یکتا از ویژگی انتخاب‌شده، یک قانون می‌سازد. این الگوریتم بسیار ساده و سریع است؛ ابتدا ویژگی‌های پیوسته را گسسته کرده، برای هر ویژگی یک جدول تقاطعی^{۷۲} با متغیر هدف تشکیل می‌دهد و در نهایت ویژگی‌ای را برای ساخت قانون انتخاب می‌کند که کمترین خطای کل را داشته باشد. این روش غالباً به عنوان یک مدل پایه^{۷۳} سریع برای ارزیابی و مقایسه با مدل‌های پیچیده‌تر مورد استفاده قرار می‌گیرد.

⁷⁰Trade-off

⁷¹One Rule

⁷²Cross table

⁷³Baseline

پوشش متوالی (Sequential Covering)

پوشش متوالی یک رویه است که به صورت مکرر یک قانون استخراج می‌کند، نقاط داده‌ای که توسط آن قانون پوشش داده شده‌اند را از مجموعه حذف می‌نماید و این فرآیند را روی داده‌های باقیمانده تکرار می‌کند تا تمام داده‌ها به طور کامل پوشش داده شوند. این استراتژی که به استراتژی «جداسازی و غلبه»^{۷۴} نیز معروف است، پایه و اساس بسیاری از الگوریتم‌های پیشرفته‌تر از جمله RIPPER^{۷۵} را تشکیل می‌دهد که در آن از تکنیک هرس کردن^{۷۶} برای بهینه‌سازی و کاهش پیچیدگی لیست قوانین استفاده می‌شود.

لیست‌های قوانین بیزی (Bayesian Rule Lists / BRL)

این رویکرد مدرن ترکیبی از دو مرحله محاسباتی است: ابتدا الگوهای پرتکرار^{۷۷} از داده‌ها با استفاده از الگوریتم‌هایی نظیر Apriori یا FP-Growth استخراج می‌شوند و سپس، از آمار بیزی برای یادگیری یک لیست تصمیم بهینه از میان الگوهای استخراج‌شده استفاده می‌گردد. هدف اصلی الگوریتم بیزی، یافتن لیستی است که بالاترین احتمال پسین را داشته باشد؛ بدین معنا که مدل نهایی علاوه بر توصیف دقیق داده‌ها، بر اساس مفروضات پیشین دارای قوانینی کوتاه و ساختاری منسجم و مختصر باشد.

۴-۴-۳ مزایا و معایب قوانین تصمیم

ارزیابی جامع قوانین تصمیم مستلزم بررسی همزمان نقاط قوت و ضعف آن‌هاست. از منظر مزایا، قوانین «اگر-آنگاه» بسیار تفسیرپذیر و سریع هستند. آن‌ها قادرند مدل‌هایی با قدرت بیان برابر با درختان تصمیم تولید کنند، اما با ساختاری فشرده‌تر و بدون درگیر شدن با مشکل تکرار زیردرخت‌ها.

همچنین، این مدل‌ها نسبت به تبدیلات یکنوای داده‌ها و حضور داده‌های پرت بسیار مقاوم بوده و به طور ذاتی مدل‌های پراکنده^{۷۸} تولید می‌کنند که تنها از ویژگی‌های کاملاً مرتبط بهره می‌برند.

با این وجود، از منظر معایب، تمرکز عمده ادبیات این حوزه بر روی مسائل طبقه‌بندی است و این مدل‌ها در کاربردهای رگرسیون (به دلیل نیاز به گسسته‌سازی متغیر هدف) با کاهش دقت مواجه می‌شوند. علاوه بر این، وابستگی شدید آن‌ها به ویژگی‌های طبقه‌ای، فرآیند پیش‌پردازش و آماده‌سازی داده‌ها را به شدت پیچیده می‌سازد. در نهایت، این مدل‌ها نیز همانند درختان تصمیم، در توصیف روابط خطی پیوسته بسیار ضعیف عمل کرده و پیش‌بینی‌های آن‌ها به توابع پله‌ای گسسته محدود می‌شود.

۵-۳ سایر مدل‌های تفسیرپذیر

فهرست مدل‌های تفسیرپذیر دائماً در حال رشد بوده و اندازه دقیق آن نامشخص است. این فهرست شامل مدل‌های ساده‌ای نظیر مدل‌های خطی، درختان تصمیم و بیز ساده^{۷۹} می‌شود، اما مدل‌های پیچیده‌تری را نیز در بر می‌گیرد که مدل‌های یادگیری

⁷⁴Separate-and-conquer

⁷⁵Repeated Incremental Pruning to Produce Error Reduction

⁷⁶Pruning

⁷⁷Frequent patterns

⁷⁸Sparse models

⁷⁹Naive Bayes

ماشین غیر قابل تفسیر را ترکیب یا اصلاح می کنند تا تفسیر پذیری آن ها را افزایش دهند. مقالات مرتبط با نوع دوم با سرعت بسیار بالایی در حال انتشار هستند و پیگیری مستمر این پیشرفت ها دشوار است. در این بخش، به عنوان نمونه تنها به بررسی طبقه بند بیز ساده و روش k همسایه نزدیک^{۸۰} پرداخته می شود.

۳-۵-۱ طبقه بند بیز ساده (Naive Bayes Classifier)

طبقه بند بیز ساده از قضیه بیز برای محاسبه احتمالات شرطی^{۸۱} استفاده می کند. این مدل برای هر ویژگی، احتمال تعلق به یک کلاس را بر اساس مقدار آن ویژگی محاسبه می نماید. طبقه بند بیز ساده، احتمالات کلاس را برای هر ویژگی به صورت کاملاً مستقل محاسبه می کند که این امر معادل با یک فرض قوی (یا همان «ساده لوحانه»^{۸۲}) مبنی بر استقلال ویژگی ها از یکدیگر است. بیز ساده یک مدل احتمال شرطی است و احتمال یک کلاس مانند C_k را به صورت زیر مدل سازی می کند:

$$P(C_k|x) = \frac{1}{Z} P(C_k) \prod_{i=1}^n P(x_i|C_k)$$

در این معادله، عبارت Z یک پارامتر مقیاس بندی^{۸۳} است که تضمین می کند مجموع احتمالات برای تمام کلاس ها برابر با ۱ باشد (در غیر این صورت، این مقادیر از نظر ریاضی احتمال محسوب نمی شوند). احتمال شرطی یک کلاس برابر است با احتمال پایه آن کلاس ضربدر احتمال هر ویژگی به شرط آن کلاس، که در نهایت توسط Z نرمال سازی می شود. این فرمول مستقیماً از قضیه بیز استخراج می گردد. دلیل تفسیر پذیری بودن مدل بیز ساده، استفاده از همین فرض استقلال ویژگی هاست. این مدل در سطح ماژولار کاملاً قابل تفسیر است؛ زیرا به لطف احتمالات شرطی، میزان مشارکت و تأثیر دقیق هر ویژگی در پیش بینی یک کلاس مشخص، بسیار واضح و شفاف است.

۳-۵-۲ روش k همسایه نزدیک (K-Nearest Neighbors / KNN)

روش k همسایه نزدیک می تواند هم برای مسائل رگرسیون و هم برای طبقه بندی مورد استفاده قرار گیرد و از نزدیک ترین همسایه های یک نقطه داده برای پیش بینی استفاده می کند.

در وظایف طبقه بندی، این روش رایج ترین کلاس را در میان همسایگان نزدیک یک نمونه به آن اختصاص می دهد و در رگرسیون، میانگین خروجی همسایگان را به عنوان پیش بینی محاسبه می کند.

چالش برانگیزترین بخش های این روش، یافتن مقدار مناسب برای پارامتر k و انتخاب نحوه اندازه گیری فاصله^{۸۴} بین نمونه هاست که در نهایت مفهوم «همسایگی» را به صورت ریاضی تعریف می کند.

مدل k همسایه نزدیک با سایر مدل های تفسیر پذیر ارائه شده در این فصل تفاوت اساسی دارد، زیرا یک الگوریتم یادگیری مبتنی بر نمونه^{۸۵} است.

حال این پرسش مطرح می شود که چنین مدلی چگونه تفسیر می گردد؟ پیش از هر چیز، باید توجه داشت که هیچ پارامتری برای یادگیری در این مدل وجود ندارد، بنابراین تفسیر پذیری در سطح ماژولار در آن کاملاً بی معناست. علاوه بر این، مدل

⁸⁰k-nearest neighbors

⁸¹Conditional probabilities

⁸²Naive

⁸³Scaling parameter

⁸⁴Distance metric

⁸⁵Instance-based learning

فاقد تفسیرپذیری سراسری^{۸۶} است، زیرا ذاتاً یک مدل محلی^{۸۷} بوده و هیچ ساختار یا وزن سراسری صریحی در آن یاد گرفته نمی‌شود. با این حال، ممکن است این روش در سطح محلی قابل تفسیر باشد. برای توضیح یک پیش‌بینی، همواره می‌توان % همسایه‌ای را که برای آن پیش‌بینی خاص استفاده شده‌اند، بازیابی و بررسی کرد.

در نهایت، تفسیرپذیر بودن این مدل صرفاً به این مسئله بستگی دارد که آیا کاربر می‌تواند یک نمونه منفرد در مجموعه داده را «تفسیر» کند یا خیر. بدیهی است که اگر یک نمونه از صدها یا هزاران ویژگی تشکیل شده باشد، درک آن غیرممکن خواهد بود؛ اما اگر با تعداد محدودی از ویژگی‌ها سروکار داشته باشیم یا راهی برای کاهش ابعاد ویژگی‌های نمونه وجود داشته باشد، این روش می‌تواند تفسیرهای شهودی و مناسبی ارائه دهد.

⁸⁶Global interpretability

⁸⁷Local model

۴ روش‌های مستقل از مدل تفسیرپذیری (Model-Agnostic Methods)

جداسازی فرآیند تولید توضیح از خودِ مدل یادگیری ماشین، که تحت عنوان روش‌های تفسیر مستقل از مدل^{۸۸} شناخته می‌شود، مزایای متعددی به همراه دارد. بزرگ‌ترین مزیت این رویکرد نسبت به روش‌های وابسته به مدل^{۸۹}، انعطاف‌پذیری بالای آن است.

زمانی که روش‌های تفسیر قابلیت اعمال بر روی هر مدلی را داشته باشند، توسعه‌دهندگان در انتخاب و استفاده از مدل‌های دلخواه خود کاملاً آزاد خواهند بود. همچنین، هر ابزاری که بر پایه این تفسیرها ساخته شود، از مدل زیربنایی خود مستقل می‌گردد. از آنجا که معمولاً برای حل یک مسئله انواع مختلفی از الگوریتم‌ها ارزیابی می‌شوند، مقایسه آن‌ها از منظر تفسیرپذیری با استفاده از روش‌های مستقل از مدل بسیار ساده‌تر خواهد بود؛ زیرا می‌توان یک روش ارزیابی یکسان را برای تمامی مدل‌ها به کار برد.

در نقطه مقابل، یکی از جایگزین‌های این روش، استفاده انحصاری از مدل‌های ذاتاً تفسیرپذیر است. این رویکرد غالباً با یک نقطه ضعف بزرگ همراه است: افت عملکرد پیش‌بینی در مقایسه با مدل‌های پیچیده‌تر و همچنین محدود شدن توسعه‌دهنده به دسته‌ای خاص از مدل‌ها. جایگزین دیگر، استفاده از روش‌های تفسیر وابسته به مدل است که آن نیز کاربر را به یک معماری خاص محدود کرده و تغییر مدل را در آینده بسیار دشوار می‌سازد.

به طور کلی، یک سیستم مطلوب برای تولید توضیحات مستقل از مدل، باید دارای سه ویژگی اساسی باشد: نخست، انعطاف‌پذیری مدل^{۹۰}؛ به این معنا که روش تفسیر بتواند با هر مدل یادگیری ماشین، از جنگل‌های تصادفی^{۹۱} گرفته تا شبکه‌های عصبی عمیق^{۹۲}، به درستی کار کند. ویژگی دوم، انعطاف‌پذیری توضیح^{۹۳} است؛ بدین معنا که کاربر به یک قالب خاص از توضیح محدود نشود، چرا که در برخی موارد ممکن است یک فرمول خطی کارآمدتر باشد و در موارد دیگر، یک نمودار بصری برای نمایش اهمیت ویژگی‌ها. و در نهایت، انعطاف‌پذیری نمایش^{۹۴}؛ این ویژگی سیستم تفسیر را قادر می‌سازد تا برای ارائه توضیحات، از نمایشی متفاوت برای ویژگی‌ها نسبت به آنچه در خودِ مدل استفاده شده است، بهره‌بردار. به عنوان مثال، برای یک دسته‌بند متنی^{۹۵} که ورودی‌های آن از جنس بردارهای انتزاعی تعبیه کلمات^{۹۶} است، ممکن است ترجیح داده شود تا توضیحات بر اساس «حضور یا عدم حضور کلمات منفرد» ارائه گردد تا برای انسان قابل فهم‌تر باشد.

۴-۱ نمودار وابستگی جزئی (PDP)

نمودار وابستگی جزئی^{۹۷} اثر حاشیه‌ای^{۹۸} یک یا دو ویژگی را بر خروجی پیش‌بینی شده یک مدل یادگیری ماشین نشان می‌دهد. یک نمودار وابستگی جزئی می‌تواند مشخص کند که آیا رابطه بین متغیر هدف و یک ویژگی، خطی، یکنواخت یا پیچیده‌تر است.

⁸⁸Model-Agnostic Interpretation Methods

⁸⁹Model-Specific Methods

⁹⁰Model Flexibility

⁹¹Random Forests

⁹²Deep Neural Networks

⁹³Explanation Flexibility

⁹⁴Representation Flexibility

⁹⁵Text Classifier

⁹⁶Word Embedding Vectors

⁹⁷Partial Dependence Plot (PDP)

⁹⁸Marginal effect

تابع وابستگی جزئی برای رگرسیون به صورت زیر تعریف می‌شود:

$$\hat{f}_{x_S}(x_S) = E_{x_C} [\hat{f}(x_S, x_C)] = \int \hat{f}(x_S, x_C) dP(x_C)$$

در این رابطه، x_S ویژگی‌هایی هستند که تابع وابستگی جزئی باید برای آن‌ها رسم شود و x_C سایر ویژگی‌های استفاده‌شده در مدل یادگیری ماشین $\hat{f}$ هستند. معمولاً تنها یک یا دو ویژگی در مجموعه S قرار می‌گیرند؛ یعنی همان ویژگی‌هایی که می‌خواهیم اثر آن‌ها را بر پیش‌بینی بدانیم. بردارهای ویژگی x_S و x_C در ترکیب با یکدیگر، کل فضای ویژگی x را تشکیل می‌دهند. نمودار وابستگی جزئی با حاشیه‌سازی^{۹۹} خروجی مدل یادگیری ماشین بر روی توزیع ویژگی‌های مجموعه C عمل می‌کند، به طوری که تابع نهایی تنها رابطه بین ویژگی‌های مورد نظر در مجموعه S (با در نظر گرفتن تعاملات آن‌ها با سایر ویژگی‌ها) و خروجی پیش‌بینی شده را نشان می‌دهد.

تابع جزئی $\hat{f}_{x_S}$ از طریق محاسبه میانگین‌ها در داده‌های آموزشی، که به روش مونت کارلو^{۱۰۰} نیز معروف است، به صورت زیر تخمین زده می‌شود:

$$\hat{f}_{x_S}(x_S) = \frac{1}{n} \sum_{i=1}^n \hat{f}(x_S, x_C^{(i)})$$

این تابع به ما می‌گوید که به ازای مقادیر مشخصی از ویژگی‌های S ، میانگین اثر حاشیه‌ای بر روی پیش‌بینی چقدر است. در این فرمول، $x_C^{(i)}$ مقادیر واقعی ویژگی‌هایی در مجموعه داده هستند که مورد بررسی ما نیستند و n تعداد نمونه‌ها در مجموعه داده است.

یکی از مفروضات اساسی PDP این است که ویژگی‌های C با ویژگی‌های S همبستگی ندارند. اگر این فرض نقض شود، میانگین‌های محاسبه‌شده شامل نقاط داده‌ای خواهند شد که بسیار نامحتمل یا حتی غیرممکن هستند. برای مسائل طبقه‌بندی که مدل خروجی احتمالات را برمی‌گرداند، این نمودار احتمال یک کلاس خاص را با توجه به مقادیر مختلف ویژگی‌های S نشان می‌دهد (برای مدیریت کلاس‌های چندگانه، می‌توان برای هر کلاس یک خط یا نمودار جداگانه رسم کرد). در نهایت، نمودار وابستگی جزئی یک روش سراسری^{۱۰۱} است، زیرا تمامی نمونه‌ها را در نظر گرفته و بیانیه‌ای درباره رابطه کلی یک ویژگی با خروجی پیش‌بینی شده ارائه می‌دهد.

۴-۱-۱ ویژگی‌های رسته‌ای و مثال‌های کاربردی

برای ویژگی‌های رسته‌ای^{۱۰۲}، محاسبه وابستگی جزئی بسیار ساده است. برای هر یک از رسته‌ها، مقدار PDP با وادار کردن تمامی نمونه‌های داده به داشتن همان دسته محاسبه می‌شود.

به عنوان مثال، اگر در مجموعه داده کرایه دوچرخه بخواهیم نمودار وابستگی جزئی را برای ویژگی «فصل» رسم کنیم، چهار عدد به دست می‌آوریم.

برای محاسبه مقدار «تابستان»، فصل تمامی نمونه‌ها را با «تابستان» جایگزین کرده و میانگین پیش‌بینی‌ها را محاسبه می‌کنیم. در عمل، مجموعه ویژگی‌های S معمولاً تنها شامل یک یا حداکثر دو ویژگی است، زیرا یک ویژگی منجر به نمودار دوبعدی و دو ویژگی منجر به نمودار سه‌بعدی می‌شود و تجسم فراتر از آن بسیار دشوار است.

⁹⁹ Marginalizing

¹⁰⁰ Monte Carlo method

¹⁰¹ Global method

¹⁰² Categorical features

برای درک بهتر این مفاهیم، می‌توان یک مدل جنگل تصادفی^{۱۰۳} را برای پیش‌بینی تعداد دوچرخه‌های کرایه‌ای برآزش داد و از نمودار وابستگی جزئی برای تجسم روابط یادگرفته‌شده (مانند تأثیر دما، رطوبت و سرعت باد) استفاده کرد.

این بررسی‌ها می‌تواند نشان دهد که با افزایش دما تا مرز مشخصی تقاضا افزایش می‌یابد، یا رطوبت بالا اثر بازدارنده‌ای بر کرایه دوچرخه دارد. این رویکرد به طور مشابه برای پیش‌بینی احتمالات پزشکی، مانند ارزیابی خطر بیماری‌ها بر اساس سن یا سابقه پزشکی، و حتی بررسی اثرات تعاملی دو متغیر به طور همزمان نیز کاربرد گسترده‌ای دارد.

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from sklearn.ensemble import RandomForestRegressor
5 from sklearn.inspection import PartialDependenceDisplay
6 from sklearn.model_selection import train_test_split
7
8 # 1. Create synthetic bike-sharing dataset
9 np.random.seed(42)
10 n = 1000
11 temperature = np.random.uniform(0, 35, n)
12 humidity = np.random.uniform(20, 100, n)
13 windspeed = np.random.uniform(0, 35, n)
14
15 # Simulated target: number of rented bikes
16 bikes = (
17     3000
18     + temperature * 90
19     - humidity * 8
20     - windspeed * 12
21     + np.random.normal(0, 150, n)
22 )
23
24 df = pd.DataFrame({
25     "temperature": temperature,
26     "humidity": humidity,
27     "windspeed": windspeed,
28     "bikes": bikes
29 })
30
31 # 2. Train model
32 X = df[["temperature", "humidity", "windspeed"]]
```

¹⁰³Random forest

```

۳۳ y = df["bikes"]
۳۴
۳۵ X_train, X_test, y_train, y_test = train_test_split(
۳۶     X, y,
۳۷     test_size=0.2,
۳۸     random_state=42
۳۹ )
۴۰
۴۱ model = RandomForestRegressor(
۴۲     n_estimators=100,
۴۳     random_state=42
۴۴ )
۴۵
۴۶ model.fit(X_train, y_train)
۴۷
۴۸ # 3. Plot PDP (Partial Dependence Plot)
۴۹ fig, ax = plt.subplots(figsize=(12, 4))
۵۰
۵۱ features = ["temperature", "humidity", "windspeed"]
۵۲
۵۳ PartialDependenceDisplay.from_estimator(
۵۴     model,
۵۵     X_train,
۵۶     features=features,
۵۷     kind="average",
۵۸     ax=ax
۵۹ )
۶۰
۶۱ fig.suptitle("PDP Example", fontsize=22)
۶۲ plt.tight_layout()
۶۳ plt.show()

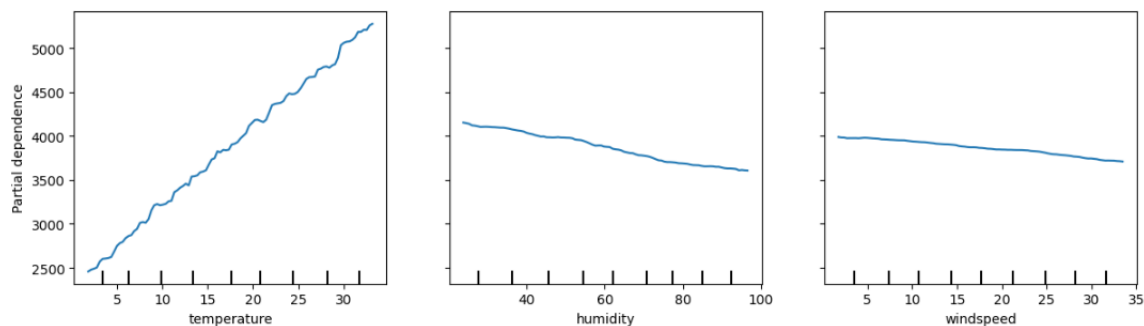
```

کد پایتون فوق یک پیاده‌سازی عملی از مفاهیم مطرح شده است و به سه بخش اصلی تقسیم می‌شود:

در این مثال، ابتدا با استفاده از کتابخانه numpy یک مجموعه داده مصنوعی شامل ۱۰۰۰ نمونه تولید می‌شود. سه ویژگی دما، رطوبت و سرعت باد به صورت تصادفی ایجاد شده و سپس متغیر هدف، یعنی تعداد دوچرخه‌های کرایه‌شده، بر اساس یک رابطه ریاضی به همراه مقداری نویز تصادفی محاسبه می‌شود تا داده‌ها به شرایط واقعی نزدیک‌تر باشند. پس از آن، تمامی ویژگی‌ها و متغیر هدف در قالب یک DataFrame ذخیره می‌شوند.

در گام بعد، داده‌ها به دو بخش آموزش و آزمون با نسبت ۸۰ به ۲۰ تقسیم شده و یک مدل Random Forest Regressor شامل ۱۰۰ درخت تصمیم روی داده‌های آموزشی آموزش داده می‌شود. این مدل با یادگیری روابط میان متغیرهای آب‌وهوایی و تعداد دوچرخه‌های کرایه‌شده، قادر به پیش‌بینی مقدار متغیر هدف خواهد بود.

پس از آموزش مدل، برای تفسیر رفتار آن از نمودارهای وابستگی جزئی (PDP) استفاده می‌شود. به این منظور، ابزار PartialDependenceDisplay برای رسم نمودارهای وابستگی جزئی مربوط به ویژگی‌های دما، رطوبت و سرعت باد به کار گرفته می‌شود. در این فرآیند، تنظیم پارامتر `kind="average"` باعث می‌شود نمودارها میانگین اثر حاشیه‌ای هر ویژگی بر پیش‌بینی‌های مدل را نمایش دهند. بدین ترتیب می‌توان مشاهده کرد که با تغییر هر یک از ویژگی‌ها و در نظر گرفتن اثر سایر متغیرها به صورت میانگین، خروجی مدل چگونه تغییر می‌کند.



شکل ۳: نمای خروجی نمودار PDP

۲-۱-۴ مزایا و معایب

محاسبه و درک نمودارهای وابستگی جزئی بسیار شهودی و ساده است؛ این نمودار به وضوح نشان می‌دهد که با تغییر یک ویژگی خاص، میانگین پیش‌بینی در مجموعه داده چگونه تغییر می‌کند و پیاده‌سازی آن نیز آسان است.

علاوه بر این، محاسبات این نمودار دارای نوعی تفسیر علی^{۱۰۴} برای مدل است، زیرا با مداخله بر روی یک ویژگی، تغییرات در پیش‌بینی مدل (و نه لزوماً در دنیای واقعی) اندازه‌گیری می‌شود. با این وجود، این روش با محدودیت‌هایی نیز روبروست. حداکثر تعداد واقعی ویژگی‌های قابل بررسی در یک تابع وابستگی جزئی، به دلیل محدودیت‌های تجسم، دو عدد است. همچنین، برخی از نمودارهای PD توزیع واقعی ویژگی‌ها را نشان نمی‌دهند که می‌تواند منجر به تفسیر بیش از حد در مناطقی شود که داده‌ای وجود ندارد؛ هرچند این مشکل با افزودن یک هیستوگرام یا نشانگرهای روی محور (Rug) قابل حل است.

بزرگترین چالش PD، فرض استقلال ویژگی‌هاست. اگر ویژگی مورد بررسی با سایر ویژگی‌ها همبستگی داشته باشد، مدل داده‌های مصنوعی و نامحتملی در توزیع حاشیه‌ای^{۱۰۵} ایجاد می‌کند (مثلاً فرض وجود فردی با دو متر قد اما وزن کمتر از ۵۰ کیلوگرم). برای رفع این مشکل، استفاده از نمودارهای اثرات محلی انباشته^{۱۰۶} پیشنهاد می‌شود که به جای توزیع حاشیه‌ای، با توزیع شرطی کار می‌کنند. در نهایت، PDP تنها میانگین اثرات حاشیه‌ای را نشان می‌دهد و این امر ممکن است سبب پنهان ماندن اثرات ناهمگون^{۱۰۷} شود. برای مثال، اگر تأثیر یک ویژگی برای نیمی از داده‌ها مثبت و برای نیمی دیگر منفی باشد، خط PD ممکن است به صورت افقی (بی‌اثر) ظاهر شود. در چنین مواردی رسم منحنی‌های امید ریاضی شرطی فردی

¹⁰⁴Causal interpretation

¹⁰⁵Marginal distribution

¹⁰⁶Accumulated Local Effect (ALE) plots

¹⁰⁷Heterogeneous effects

می‌تواند اثرات ناهمگون پنهان را آشکار سازد.

۲-۴ مدل جایگزین سراسری (Global Surrogate)

یک مدل جایگزین سراسری، یک مدل تفسیرپذیر است که برای تقریب زدن پیش‌بینی‌های یک مدل جعبه‌سیاه^{۱۰۸} آموزش داده می‌شود. با تفسیر این مدل جایگزین، می‌توانیم نتیجه‌گیری‌هایی درباره رفتار مدل جعبه‌سیاه به دست آوریم؛ به عبارت دیگر، حل چالش تفسیرپذیری یادگیری ماشین با استفاده از الگوریتم‌های یادگیری ماشین بیشتر!

۱-۲-۴ مبانی نظری و ریاضیات

مدل‌های جایگزین در مهندسی نیز کاربرد گسترده‌ای دارند: زمانی که اندازه‌گیری یک خروجی مورد نظر پرهزینه، زمان‌بر یا دشوار باشد (مثلاً خروجی یک شبیه‌سازی کامپیوتری پیچیده)، از یک مدل جایگزین سریع و ارزان به جای آن استفاده می‌شود. تفاوت اصلی مدل‌های جایگزین در حوزه یادگیری ماشین تفسیرپذیر این است که مدل پایه یک مدل یادگیری ماشین است (نه یک شبیه‌سازی فیزیکی) و مدل جایگزین باید حتماً تفسیرپذیر باشد. ایده مدل‌های جایگزین در ادبیات موضوع با نام‌های دیگری نظیر مدل تقریب^{۱۰۹}، فرامدل^{۱۱۰}، مدل سطح پاسخ^{۱۱۱} و شبیه‌ساز^{۱۱۲} نیز شناخته می‌شود. از نظر تئوری، هدف این است که تابع پیش‌بینی مدل جعبه‌سیاه (f) را تا حد امکان توسط یک تابع پیش‌بینی جایگزین (g) تقریب بزنیم، با این قید سخت‌گیرانه که g تفسیرپذیر باشد. برای تابع g می‌توان از هر مدل تفسیرپذیری استفاده کرد؛ به عنوان مثال یک مدل خطی به فرم $g(x) = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$ یا یک درخت تصمیم به فرم $g(x) = \sum_{m=1}^M c_m I\{x \in R_m\}$.

۲-۲-۴ مراحل ساخت و ارزیابی مدل جایگزین

آموزش یک مدل جایگزین یک رویکرد کاملاً مستقل از مدل^{۱۱۳} است، زیرا نیازی به اطلاعات از ساختار درونی مدل جعبه‌سیاه ندارد و تنها دسترسی به داده‌ها و تابع پیش‌بینی کفایت می‌کند.

برای ساخت این مدل، ابتدا یک مجموعه داده (X) انتخاب می‌شود که می‌تواند همان داده‌های آموزش مدل اصلی، داده‌های جدیدی از همان توزیع، یا حتی یک شبکه از نقاط داده‌ای باشد. سپس پیش‌بینی‌های مدل جعبه‌سیاه برای این داده‌ها استخراج می‌گردد. در گام بعدی، یک مدل تفسیرپذیر (مانند مدل خطی یا درخت تصمیم) انتخاب شده و بر روی مجموعه داده X و پیش‌بینی‌های به‌دست‌آمده آموزش داده می‌شود. در نهایت، میزان تطابق پیش‌بینی‌های مدل جایگزین با مدل جعبه‌سیاه اندازه‌گیری شده و مدل جایگزین تفسیر می‌گردد. یکی از معیارهای استاندارد برای اندازه‌گیری این تطابق، معیار R^2 است:

$$R^2 = 1 - \frac{SSE}{SST} = 1 - \frac{\sum_{i=1}^n (\hat{y}_*^{(i)} - \hat{y}^{(i)})^2}{\sum_{i=1}^n (\hat{y}^{(i)} - \bar{\hat{y}})^2}$$

که در آن $\hat{y}_*^{(i)}$ پیش‌بینی مدل جایگزین، $\hat{y}^{(i)}$ پیش‌بینی مدل جعبه‌سیاه و $\bar{\hat{y}}$ میانگین پیش‌بینی‌های مدل جعبه‌سیاه است. معیار

¹⁰⁸Black box model

¹⁰⁹Approximation model

¹¹⁰Metamodel

¹¹¹Response surface model

¹¹²Emulator

¹¹³Model-agnostic

R^2 نشان‌دهنده درصدی از واریانس است که توسط مدل جایگزین توضیح داده می‌شود. اگر این مقدار نزدیک به ۱ باشد، مدل تفسیرپذیر رفتار مدل جعبه‌سیاه را به خوبی تقریب می‌زند و اگر نزدیک به صفر باشد، در توضیح مدل جعبه‌سیاه شکست خورده است. شایان ذکر است که عملکرد پیش‌بینی مدل جعبه‌سیاه (در مقایسه با واقعیت) نقشی در آموزش مدل جایگزین ندارد، زیرا تفسیر مدل جایگزین صرفاً بیانیه‌هایی درباره خود «مدل» ارائه می‌دهد، نه دنیای واقعی. البته اگر مدل جعبه‌سیاه عملکرد ضعیفی داشته باشد، تفسیر مدل جایگزین آن نیز عملاً فاقد ارزش خواهد بود.

۳-۲-۴ مثال‌های کاربردی

برای نشان دادن کارایی مدل‌های جایگزین، می‌توان یک مثال رگرسیون و یک مثال طبقه‌بندی را در نظر گرفت. در یک سناریوی رگرسیون برای پیش‌بینی تعداد دوچرخه‌های کرایه‌ای روزانه، ابتدا یک ماشین بردار پشتیبان^{۱۱۴} آموزش داده می‌شود. از آنجا که این مدل به راحتی قابل تفسیر نیست، یک درخت تصمیم CART به عنوان مدل جایگزین آموزش داده می‌شود تا رفتار آن را تقلید کند. اگر معیار R^2 برای این مدل جایگزین به عنوان مثال 0.77 باشد، نشان می‌دهد که رفتار مدل جعبه‌سیاه به شکل قابل قبولی تقریب زده شده است.

```

۱ import pandas as pd
۲ from sklearn.svm import SVR
۳ from sklearn.tree import DecisionTreeRegressor, export_text
۴ from sklearn.model_selection import train_test_split
۵ from sklearn.metrics import r2_score
۶ from sklearn.preprocessing import StandardScaler
۷
۸ # 1. Load the real Bike Sharing dataset
۹ url = "https://raw.githubusercontent.com/christophM/interpretable-ml-book/master/
    data/bike.csv"
۱۰ df = pd.read_csv(url)
۱۱
۱۲ features = ["temp", "hum", "windspeed", "season", "weekday"]
۱۳ X = df[features]
۱۴ y = df["cnt"] # Total number of rented bikes
۱۵
۱۶ X_encoded = pd.get_dummies(X, drop_first=True)
۱۷ encoded_features = X_encoded.columns #
۱۸
۱۹ # 2. Standardize features
۲۰ scaler = StandardScaler()
۲۱
۲۲ X_scaled = scaler.fit_transform(X_encoded)
۲۳ X_scaled_df = pd.DataFrame(X_scaled, columns=encoded_features)

```

¹¹⁴Support Vector Machine (SVM)

```

۲۴
۲۵ X_train, X_test, y_train, y_test = train_test_split(
۲۶     X_scaled_df, y, test_size=0.2, random_state=42
۲۷ )
۲۸
۲۹ # 3. Train the Black Box Model (Support Vector Machine - SVR)
۳۰ black_box_model = SVR(kernel='rbf', C=5000, gamma='scale')
۳۱ black_box_model.fit(X_train, y_train)
۳۲
۳۳ # 4. Extract predictions from the Black Box model
۳۴ bb_train_predictions = black_box_model.predict(X_train)
۳۵
۳۶ # 5. Train the Global Surrogate Model (Decision Tree - CART)
۳۷ surrogate_model = DecisionTreeRegressor(max_depth=3, random_state=42)
۳۸ surrogate_model.fit(X_train, bb_train_predictions)
۳۹
۴۰ # 6. Evaluate Fidelity using R-squared (R2) score
۴۱ bb_test_predictions = black_box_model.predict(X_test)
۴۲ surrogate_test_predictions = surrogate_model.predict(X_test)
۴۳
۴۴ fidelity_r2 = r2_score(bb_test_predictions, surrogate_test_predictions)
۴۵ print(f"R-squared (Fidelity of Surrogate to Black Box): {fidelity_r2:.2f}\n")
۴۶ print("-" * 50)
۴۷
۴۸ # 7. Interpret the surrogate model
۴۹ tree_rules = export_text(surrogate_model, feature_names=list(encoded_features))
۵۰ print("\nDecision rules of the Surrogate Model (approximating SVM behavior):\n")
۵۱ print(tree_rules)

```

۴-۲-۴ مزایا و معایب

روش مدل جایگزین به دلیل انعطاف‌پذیری بالا بسیار ارزشمند است؛ محققان می‌توانند از هر مدل تفسیرپذیری استفاده کنند و حتی مدل‌های جعبه‌سیاه زیربنایی را به راحتی تغییر دهند. این روش همچنین بسیار شهودی بوده و پیاده‌سازی و توضیح منطق آن به افراد غیرمتخصص آسان است.

با این حال، استفاده از این روش با چالش‌هایی نیز همراه است. محقق باید همواره در نظر داشته باشد که تمام نتیجه‌گیری‌ها تنها در مورد «مدل» انجام می‌شود و نه «داده‌های واقعی»، زیرا مدل جایگزین هرگز خروجی‌های واقعی را نمی‌بیند. علاوه بر این، تعیین حد آستانه قطعی برای معیار R^2 به منظور اطمینان از دقت تقریب (مثلاً ۸۰٪ یا ۹۹٪) همواره مبهم است.

مشکل دیگر این است که ممکن است مدل جایگزین در یک زیرمجموعه از داده‌ها تقریب بسیار خوبی داشته باشد، اما

در بخش دیگری انحراف شدیدی نشان دهد که این امر یکپارچگی تفسیر را زیر سؤال می‌برد. در نهایت، مدل جایگزین تمام مزایا و معایب مدل تفسیرپذیر انتخاب شده را به ارث می‌برد و برای پژوهشگرانی که معتقدند هیچ مدلی ذاتاً تفسیرپذیر نیست، این روش ممکن است توهم تفسیرپذیری ایجاد کند.

۳-۴ مدل جایگزین محلی (Local Surrogate / LIME)

مدل‌های جایگزین محلی^{۱۱۵}، مدل‌های تفسیرپذیری هستند که برای توضیح پیش‌بینی‌های فردی مدل‌های جعبه‌سیاه^{۱۱۶} یادگیری ماشین مورد استفاده قرار می‌گیرند. مقاله LIME (توضیحات محلی تفسیرپذیر مستقل از مدل)^{۱۱۷}، یک پیاده‌سازی ملموس از این مدل‌ها را پیشنهاد می‌دهد.

برخلاف مدل‌های جایگزین سراسری که سعی در تقریب کل مدل دارند، LIME منحصراً بر آموزش مدل‌های جایگزین محلی برای توضیح یک پیش‌بینی خاص تمرکز می‌کند.

ایده پایه بسیار شهودی است: فرض کنید تنها مدل جعبه‌سیاه را در اختیار دارید و می‌توانید با وارد کردن نقاط داده‌ای مختلف، پیش‌بینی‌های آن را بررسی کنید.

برای درک اینکه چرا مدل یک پیش‌بینی خاص را برای یک داده مشخص انجام داده است، LIME بررسی می‌کند که با ایجاد تغییرات و نوسانات کوچک در آن داده، پیش‌بینی‌های مدل چگونه تغییر می‌کند.

LIME با تولید یک مجموعه داده جدید متشکل از نمونه‌های جایگشت‌یافته^{۱۱۸} و پیش‌بینی‌های متناظر آن‌ها توسط مدل جعبه‌سیاه، یک مدل تفسیرپذیر (مانند رگرسیون خطی یا درخت تصمیم) آموزش می‌دهد. این مدل تفسیرپذیر بر اساس میزان نزدیکی نمونه‌های جدید به داده اصلی وزن‌دهی می‌شود.

مدل یادگرفته‌شده باید تقریب محلی بسیار خوبی از مدل جعبه‌سیاه باشد (که به آن وفاداری محلی^{۱۱۹} می‌گویند)، اما نیازی نیست در سطح سراسری دقیق عمل کند.

از نظر ریاضی، مدل‌های جایگزین محلی با قید تفسیرپذیری به صورت زیر بیان می‌شوند:

$$\text{explanation}(x) = \arg \min_{g \in G} L(f, g, \pi_x) + \Omega(g)$$

در این فرمول، مدل توضیح‌دهنده برای نمونه x مدلی مانند g است که زیان L را به حداقل می‌رساند. این زیان اندازه می‌گیرد که توضیحات چقدر به پیش‌بینی‌های مدل اصلی f نزدیک هستند. همزمان، پیچیدگی مدل g که با $\Omega(g)$ نشان داده می‌شود، باید پایین نگه داشته شود. G خانواده توضیحات ممکن است و معیار نزدیکی π_x اندازه همسایگی اطراف نمونه x را مشخص می‌کند. در عمل، LIME تنها بخش زیان را بهینه‌سازی می‌کند و پیچیدگی (مثلاً تعداد ویژگی‌ها) توسط کاربر تعیین می‌گردد.

مراحل آموزش مدل‌های جایگزین محلی شامل انتخاب نمونه مورد نظر، ایجاد اختلال در داده‌ها برای تولید نمونه‌های جدید، وزن‌دهی به این نمونه‌ها بر اساس مجاورت با نمونه اصلی، آموزش یک مدل تفسیرپذیر وزن‌دار و در نهایت، تفسیر مدل محلی است.

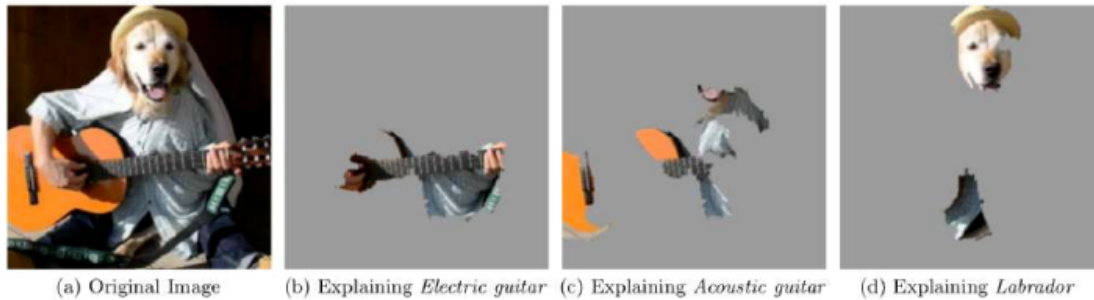
¹¹⁵Local surrogate models

¹¹⁶Black box

¹¹⁷Local Interpretable Model-agnostic Explanations

¹¹⁸Permuted samples

¹¹⁹Local fidelity



شکل ۴: نمونه حذف معنادار بخش های عکس در تفسیر LIME

۱-۳-۴ LIME برای داده های جدولی

در داده های جدولی، تولید نمونه های جدید توسط LIME با ایجاد اختلال در هر ویژگی به صورت جداگانه (با استفاده از توزیع نرمال بر اساس میانگین و انحراف معیار ویژگی ها) انجام می شود. یکی از چالش های اساسی، تعریف یک «همسایگی» معنادار است. LIME در حال حاضر از یک هسته هموارسازی نمایی^{۱۲۰} برای وزن دهی به نمونه های مصنوعی استفاده می کند که به صورت زیر تعریف می شود:

$$\pi_x(z) = \exp\left(-\frac{D(x, z)^2}{\sigma^2}\right)$$

که در آن $D(x, z)$ فاصله بین نمونه اصلی x و نمونه مصنوعی z و σ پهنای هسته^{۱۲۱} است.

پهنای هسته دامنه همسایگی را تعیین می نماید. تعیین دقیق این پارامتر بسیار حیاتی و در عین حال دشوار است، زیرا تغییر آن می تواند جهت توضیحات را کاملاً معکوس کند. این مشکل در فضاهای با ابعاد بالا شدیدتر می شود، زیرا مقایسه فواصل در ابعاد مختلف (ویژگی های متفاوت) ممکن است فاقد منطق ریاضی یکپارچه باشد.

۲-۳-۴ LIME برای متن و تصویر

تولید داده های جدید برای متون و تصاویر تفاوت های ساختاری دارد. در متن، تغییرات با حذف تصادفی کلمات از متن اصلی ایجاد می شوند و هر ویژگی به صورت باینری (حضور یا عدم حضور کلمه) نمایش داده می شود. وزن هر نمونه جدید بر اساس نسبت کلمات حفظ شده محاسبه می گردد.

در مورد تصاویر، به جای تغییر پیکسل های منفرد که تأثیر ناچیزی دارند، تصویر به «ابرپیکسل ها»^{۱۲۲} (پیکسل های به هم پیوسته با رنگ های مشابه) تقسیم می شود. LIME با خاموش کردن یا جایگزین کردن این ابرپیکسل ها (مثلاً با رنگ خاکستری) نمونه های جدید را تولید می کند تا نشان دهد کدام بخش از تصویر بیشترین تأثیر را در پیش بینی داشته است.

¹²⁰Exponential smoothing kernel

¹²¹Kernel Width

¹²²Superpixels

در ادامه، یک مثال عملی و جامع از پیاده‌سازی الگوریتم LIME با استفاده از زبان پایتون و کتابخانه رسمی lime آورده شده است. در این قطعه کد، ابتدا یک مدل جعبه‌سیاه (جنگل تصادفی) بر روی مجموعه داده معروف زنبق (Iris) آموزش داده می‌شود و سپس نحوه تصمیم‌گیری مدل برای یک نمونه‌ی خاص، توسط LIME توضیح و تفسیر می‌گردد.

```

۱ import numpy as np
۲ import matplotlib.pyplot as plt
۳ from sklearn.datasets import load_iris
۴ from sklearn.model_selection import train_test_split
۵ from sklearn.ensemble import RandomForestClassifier
۶ from lime.lime_tabular import LimeTabularExplainer
۷
۸ # 1. Load dataset
۹ iris = load_iris()
۱۰
۱۱ X = iris.data
۱۲ y = iris.target
۱۳
۱۴ feature_names = iris.feature_names
۱۵ class_names = iris.target_names
۱۶
۱۷ X_train, X_test, y_train, y_test = train_test_split(
۱۸     X, y,
۱۹     test_size=0.2,
۲۰     random_state=42
۲۱ )
۲۲
۲۳ # 2. Train model
۲۴ model = RandomForestClassifier(random_state=42)
۲۵ model.fit(X_train, y_train)
۲۶ # Accuracy
۲۷ accuracy = model.score(X_test, y_test)
۲۸ print("Model Accuracy:", accuracy)
۲۹
۳۰ # 3. Create LIME explainer
۳۱ explainer = LimeTabularExplainer(
۳۲     training_data=X_train,
۳۳     feature_names=feature_names,
۳۴     class_names=class_names,
۳۵     mode='classification'
۳۶ )

```

```

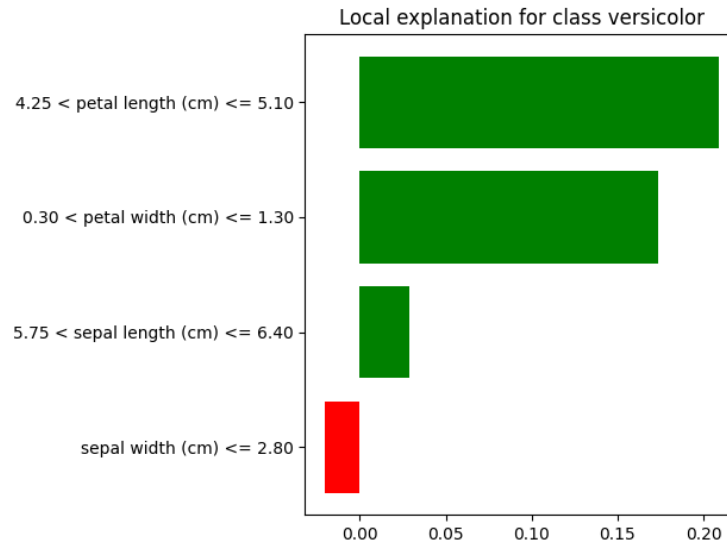
۳۷
۳۸ # 4. Select sample
۳۹ sample_index = 0
۴۰ sample = X_test[sample_index]
۴۱
۴۲ # 5. Model prediction
۴۳ prediction = model.predict([sample])[0]
۴۴
۴۵ print("\nPredicted class:", class_names[prediction])
۴۶ print("Actual class:", class_names[y_test[sample_index]])
۴۷
۴۸ # 6. Generate explanation
۴۹ exp = explainer.explain_instance(
۵۰     data_row=sample,
۵۱     predict_fn=model.predict_proba,
۵۲     num_features=4
۵۳ )
۵۴
۵۵ print("\nLIME Explanation:")
۵۶ for feature, weight in exp.as_list():
۵۷     print(f"{feature}: {weight:.4f}")
۵۸
۵۹ # 7. Plot
۶۰ fig = exp.as_pyplot_figure()
۶۱ plt.tight_layout()
۶۲ plt.show()

```

در این پیاده‌سازی، فرآیند کار با آماده‌سازی داده‌ها و آموزش مدل آغاز می‌شود؛ بدین صورت که ابتدا داده‌ها به دو بخش آموزش و آزمون تقسیم شده و یک مدل جنگل تصادفی روی آن‌ها برازش می‌شود که این مدل در واقع همان «مدل جعبه سیاه» مورد بحث است.

در گام بعدی، کلاس LimeTabularExplainer با استفاده از داده‌های آموزشی مقداردهی می‌شود تا الگوریتم LIME بتواند با استخراج میانگین و انحراف معیار ویژگی‌ها، داده‌های مصنوعی جدید را از یک توزیع نرمال نمونه‌برداری کند.

پس از انتخاب یک نمونه مشخص برای تفسیر، مهم‌ترین بخش کد یعنی فراخوانی متد explain_instance اجرا می‌گردد. این متد با دریافت نمونه مورد نظر، تابع پیش‌بینی احتمالاتِ مدل اصلی (predict_proba) و تعداد ویژگی‌های مد نظر برای نمایش، محاسبات محلی را انجام می‌دهد. در نهایت، خروجی‌های الگوریتم که نشان‌دهنده وزن‌های محلی استخراج شده از مدل جایگزین (نظیر رگرسیون خطی) هستند، در قالب یک نمودار میله‌ای مصورسازی می‌شوند تا به وضوح نشان دهند کدام مقادیر ویژگی‌ها تأثیر مثبت یا منفی در انتخاب کلاس پیش‌بینی شده برای آن نمونه خاص داشته‌اند.



شکل ۵: تفسیر محلی پیش‌بینی مدل با استفاده از الگوریتم LIME برای کلاس versicolor. این نمودار میله‌ای نشان می‌دهد که چگونه محدوده‌های خاصی از مقادیر ویژگی‌ها در تصمیم‌گیری مدل برای طبقه‌بندی یک نمونه‌ی مشخص نقش داشته‌اند. میله‌های سبزرنگ نشان‌دهنده‌ی شرایطی هستند که تأثیر مثبت بر پیش‌بینی این کلاس داشته‌اند؛ به عنوان مثال، قرار داشتن طول گلبرگ (petal length) در بازه‌ی [4.25, 5.10] بیشترین نقش را در هدایت مدل به سمت این کلاس ایفا کرده است. در مقابل، میله‌ی قرمز رنگ بیانگر ویژگی‌هایی است (مانند عرض کاسبرگ کوچکتر یا مساوی 2.80) که تأثیر منفی بر این پیش‌بینی داشته و احتمال تعلق نمونه به کلاس versicolor را کاهش داده‌اند.

۴-۳-۴ مزایا و معایب

LIME از انعطاف‌پذیری بالایی برخوردار است و امکان تغییر مدل جعبه‌سیاه بدون نیاز به تغییر روش تفسیر را فراهم می‌کند. این الگوریتم می‌تواند از ویژگی‌هایی متفاوت با ویژگی‌های مدل اصلی برای ارائه توضیحات استفاده کند (مثلاً استفاده از کلمات قابل فهم به جای بردارهای انتزاعی).

توضیحات آن به ویژه هنگام استفاده از مدل‌های خطی کوتاه، بسیار کاربرپسند و متضاد^{۱۲۳} هستند که برای افراد غیرمتخصص مناسب است. با این وجود، LIME در محیط‌های نیازمند به تطابق قانونی^{۱۲۴} کارایی کافی ندارد، زیرا توضیحات کاملی ارائه نمی‌دهد.

مهم‌ترین چالش حل‌نشده LIME در داده‌های جدولی، تعریف صحیح همسایگی و انتخاب پهنای هسته است. علاوه بر این، روش نمونه‌برداری از توزیع گاوسی مستقل، همبستگی بین ویژگی‌ها را نادیده می‌گیرد و ممکن است نقاط داده‌ای غیرمحمول تولید کند.

همچنین، ناپایداری توضیحات یک مشکل جدی است؛ به طوری که تکرار فرآیند نمونه‌برداری می‌تواند برای یک نقطه ثابت، توضیحات متفاوتی تولید کند که این امر اعتماد به مدل را کاهش می‌دهد.

¹²³Contrastive

¹²⁴Compliance scenarios

در مجموع، LIME یک روش بسیار کمک کننده است، اما پیش از کاربرد قطعی در سیستم‌های حساس، نیازمند رفع چالش‌های اساسی است.

۴-۴ مقادیر شاپلی (Shapley Values)

پیش‌بینی یک مدل را می‌توان با فرض اینکه هر مقدار از ویژگی‌های یک نمونه به عنوان یک «بازیکن» در یک بازی عمل می‌کند و خروجی پیش‌بینی به منزله «پرداختی»^{۱۲۵} آن بازی است، توضیح داد. مقدار شاپلی که ریشه در نظریه بازی‌های ائتلافی^{۱۲۶} دارد، روشی را ارائه می‌دهد که به کمک آن می‌توان این پرداختی (یا همان پیش‌بینی مدل) را به شکلی کاملاً عادلانه در میان ویژگی‌ها توزیع کرد.

۱-۴-۴ ایده کلی و منطق پایه

برای درک بهتر، فرض کنید یک مدل یادگیری ماشین برای پیش‌بینی قیمت آپارتمان آموزش داده‌اید و این مدل برای یک آپارتمان ۵۰ متری واقع در طبقه دوم که در نزدیکی یک پارک قرار دارد و ورود گربه به آن ممنوع است، قیمت ۳۰۰ هزار یورو را پیش‌بینی کرده است. اگر میانگین پیش‌بینی برای تمام آپارتمان‌ها ۳۱۰ هزار یورو باشد، هدف ما این است که توضیح دهیم هر یک از مقادیر ویژگی‌ها چقدر نسبت به این میانگین در پیش‌بینی نقش داشته‌اند. در حالی که در رگرسیون خطی به دلیل ماهیت خطی مدل، این محاسبه به سادگی از طریق ضرب وزن ویژگی در مقدار آن به دست می‌آید، در مدل‌های پیچیده‌تر به رویکرد متفاوتی نیاز است.

مقدار شاپلی، معرفی شده توسط لوید شاپلی [۳]، رویکردی است که پرداختی‌ها را بر اساس مشارکت هر بازیکن در یک ائتلاف تخصیص می‌دهد.

در اینجا، «بازی» همان وظیفه پیش‌بینی برای یک نمونه منفرد است، «سود» برابر با تفاوت پیش‌بینی واقعی و میانگین پیش‌بینی‌هاست، و «بازیکنان» مقادیر ویژگی‌های آن نمونه هستند که برای رسیدن به پیش‌بینی نهایی با یکدیگر همکاری کرده‌اند.

مقدار شاپلی برای هر ویژگی، در واقع میانگین مشارکت حاشیه‌ای^{۱۲۷} آن ویژگی در میان تمامی ائتلاف‌های ممکن از ویژگی‌هاست.

۲-۴-۴ ریاضیات و تئوری مقدار شاپلی

برای ورود به جزئیات ریاضی، ابتدا ویژگی‌های یک مدل خطی را در نظر بگیرید. در یک مدل خطی، مشارکت (ϕ_j) ویژگی j ام در پیش‌بینی $\hat{f}(x)$ برابر است با تفاوت اثر آن ویژگی و میانگین اثر آن:

$$\phi_j(\hat{f}) = \beta_j x_j - E(\beta_j X_j) = \beta_j x_j - \beta_j E(X_j)$$

¹²⁵Payout

¹²⁶Coalitional game theory

¹²⁷Average marginal contribution

اگر تمام مشارکت‌ها را با هم جمع کنیم، تفاوت بین پیش‌بینی برای داده x و میانگین پیش‌بینی‌ها به دست می‌آید:

$$\sum_{j=1}^p \phi_j(\hat{f}) = \sum_{j=1}^p (\beta_j x_j - E(\beta_j X_j)) = \hat{f}(x) - E(\hat{f}(X))$$

مقدار شاپلی این مفهوم را برای هر مدل یادگیری ماشین و از طریق یک تابع ارزش^{۱۲۸} تعمیم می‌دهد. مقدار شاپلی یک ویژگی برابر است با سهم آن در پرداختی که روی تمام ترکیبات ممکن از ویژگی‌ها وزن‌دهی و جمع شده است:

$$\phi_j(val) = \sum_{S \subseteq \{x_1, \dots, x_p\} \setminus \{x_j\}} \frac{|S|!(p - |S| - 1)!}{p!} (val(S \cup \{x_j\}) - val(S))$$

در این رابطه، S زیرمجموعه‌ای از ویژگی‌ها، x بردار ویژگی‌های نمونه مورد بررسی، و p تعداد کل ویژگی‌هاست. تابع $val_x(S)$ پیش‌بینی مدل برای ائتلاف S است که روی ویژگی‌های خارج از S حاشیه‌سازی^{۱۲۹} شده است.

مقدار شاپلی تنها روش انتساب اثری است که چهار ویژگی کارایی (Efficiency)، تقارن (Symmetry)، ویژگی بی‌اثر (Dummy) و جمع‌پذیری (Additivity) را به طور همزمان ارضا می‌کند که این موارد روی هم رفته تعریفی از یک توزیع عادلانه را شکل می‌دهند. ویژگی کارایی تضمین می‌کند که مجموع مشارکت‌ها دقیقاً برابر با تفاوت پیش‌بینی نمونه و پیش‌بینی میانگین باشد.

۳-۴-۴ تخمین مقادیر شاپلی

محاسبه دقیق مقدار شاپلی برای مدل‌هایی با ویژگی‌های متعدد به دلیل رشد نمایی تعداد ائتلاف‌ها (به میزان 2^p) عملاً غیرممکن است. برای حل این مشکل، از روش تقریب با استفاده از نمونه‌گیری مونت-کارلو^{۱۳۰} استفاده می‌شود:

$$\hat{\phi}_j = \frac{1}{M} \sum_{m=1}^M (\hat{f}(x_{+j}^m) - \hat{f}(x_{-j}^m))$$

در این تقریب، برای هر تکرار از مجموع M تکرار، یک نمونه تصادفی (z) از داده‌ها انتخاب شده و مقادیر ویژگی‌های آن با نمونه مورد نظر (x) به صورت تصادفی ترکیب می‌شوند تا دو نمونه جدید (یکی شامل ویژگی z از نمونه اصلی و دیگری شامل ویژگی z از نمونه تصادفی) ساخته شوند. تفاوت پیش‌بینی مدل جعبه‌سیاه برای این دو نمونه، مشارکت حاشیه‌ای را برای آن تکرار نشان می‌دهد و میانگین این اختلافات در نهایت به عنوان تخمین مقدار شاپلی در نظر گرفته می‌شود.

۴-۴-۴ پیاده‌سازی SHAP

این کد به منظور آموزش یک مدل یادگیری ماشین (جنگل تصادفی) و سپس تفسیر پیش‌بینی‌های آن با استفاده از رویکرد مقادیر SHAP پیاده‌سازی شده است.

```

۱ import shap
۲ import pandas as pd

```

¹²⁸Value function

¹²⁹Marginalized

¹³⁰Monte-Carlo sampling

```

۳ from sklearn.model_selection import train_test_split
۴ from sklearn.ensemble import RandomForestClassifier
۵
۶ # 1. Load sample data (using the standard 'adult' income dataset here)
۷ X, y = shap.datasets.adult()
۸ X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42)
۹
۱۰ # 2. Train the Machine Learning model
۱۱ model = RandomForestClassifier(n_estimators=100, max_depth=5, random_state=42)
۱۲ model.fit(X_train, y_train)
۱۳
۱۴ print(X.head(10))
۱۵
۱۶ # 3. Create the Explainer and calculate SHAP values
۱۷ explainer = shap.TreeExplainer(model)
۱۸ shap_values = explainer(X_test)
۱۹
۲۰ # 4. Interpretation and Plotting
۲۱ shap.plots.waterfall(shap_values[0, :, 1]) # Index 1 targets the positive class
۲۲ shap.plots.beeswarm(shap_values[:, :, 1])

```

در این فرآیند، ابتدا مجموعه داده استاندارد adult بارگذاری شده و به دو بخش آموزش (Train) و آزمون (Test) تقسیم می‌شود تا مدل روی داده‌های دیده‌نشده ارزیابی گردد.

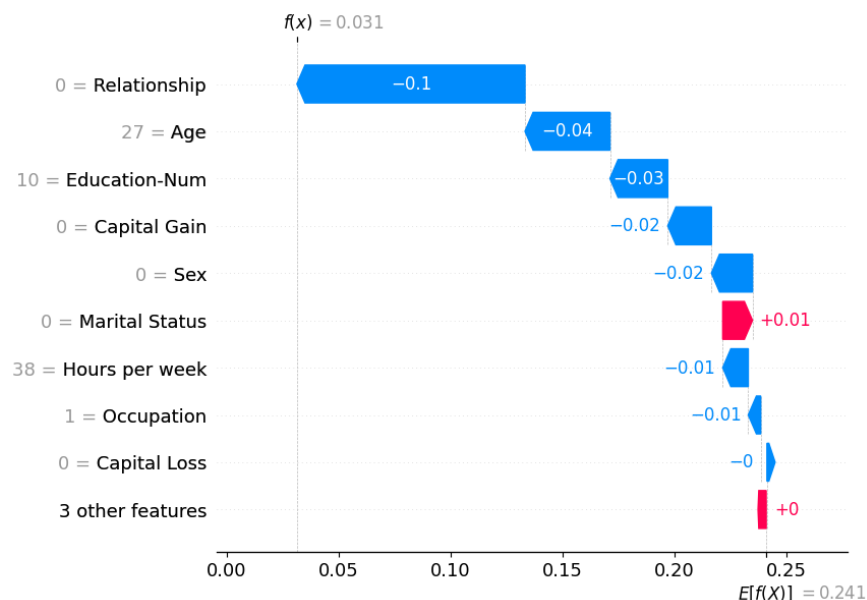
سپس یک مدل دسته‌بند جنگل تصادفی با ۱۰۰ درخت و حداکثر عمق ۵ تنظیم و روی داده‌های آموزش برازش داده می‌شود. برای تفسیر عملکرد این مدل، یک مفسر درختی^{۱۳۱} ایجاد شده و مقادیر SHAP برای داده‌های آزمون محاسبه می‌شوند؛ این مقادیر در واقع میزان سهم و تاثیر هر ویژگی را در تغییر خروجی مدل نسبت به پیش‌بینی پایه (میانگین) نشان می‌دهند.

در نهایت، با رسم نمودار آبشاری^{۱۳۲} سهم ویژگی‌ها در پیش‌بینی برای یک نمونه خاص (نمونه اول) بررسی شده و با استفاده از نمودار زنبوری^{۱۳۳}، دیدگاهی کلی از توزیع مقادیر SHAP و میزان اهمیت و جهت تاثیر تمامی ویژگی‌ها در کل مجموعه داده آزمون به دست می‌آید.

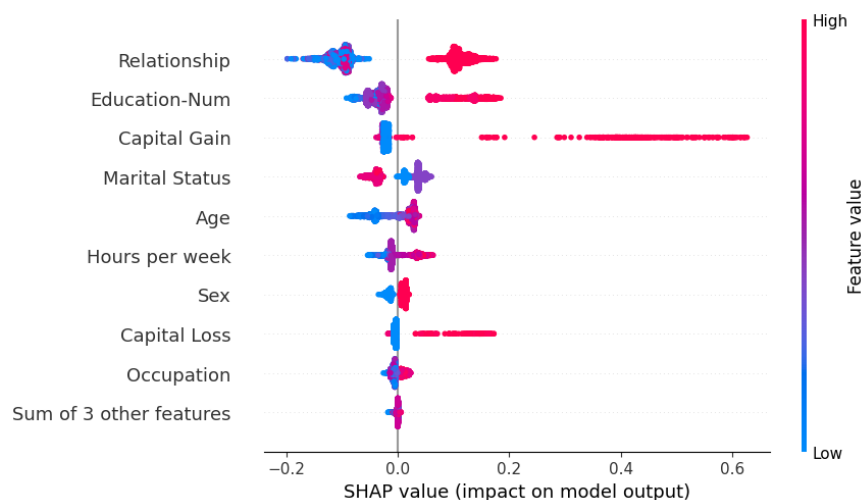
¹³¹TreeExplainer

¹³²Waterfall

¹³³Beeswarm



شکل ۶: نمودار آبشاری SHAP برای تفسیر محلی یک نمونه خاص. این نمودار نشان می‌دهد که چگونه مقادیر ویژگی‌های مختلف، خروجی مدل را از مقدار پایه یا مورد انتظار ($E[f(X)] = ۰٫۲۴۱$) به خروجی نهایی ($f(x) = ۰٫۰۳۱$) تغییر داده‌اند. فلش‌های آبی‌رنگ (مانند Relationship = 0 و Age = 27) نشان‌دهنده ویژگی‌هایی هستند که تأثیر منفی بر پیش‌بینی داشته و مقدار آن را کاهش داده‌اند. در مقابل، فلش‌های قرمز رنگ (مانند Marital Status = 0) ویژگی‌هایی با تأثیر مثبت را نشان می‌دهند که باعث افزایش مقدار پیش‌بینی شده‌اند.



شکل ۷: نمودار SHAP (از نوع Beeswarm) برای ارزیابی اهمیت سراسری ویژگی‌ها و نحوه توزیع تأثیر آن‌ها بر خروجی مدل. در این نمودار، ویژگی‌ها بر اساس میزان اهمیتشان در محور عمودی از بالا به پایین مرتب شده‌اند. محور افقی نشان‌دهنده مقدار SHAP است که میزان و جهت تأثیر (مثبت یا منفی) بر پیش‌بینی مدل را مشخص می‌کند. رنگ هر نقطه نمایانگر مقدار واقعی آن ویژگی برای یک نمونه خاص است (رنگ قرمز نشان‌دهنده مقادیر بالا و رنگ آبی نشان‌دهنده مقادیر پایین). به عنوان مثال، مقادیر بالای ویژگی Education-Num (نقاط قرمز) تأثیر مثبتی بر خروجی مدل دارند، و مقادیر بالای ویژگی Capital Gain پیش‌بینی را به شدت افزایش می‌دهند.

۵-۴-۴ مزایا و معایب

استفاده از مقادیر شاپلی با مزایا و معایب مختص به خود همراه است. از منظر نقاط قوت، این روش به دلیل داشتن پشتوانه تئوریک بسیار قدرتمند و رعایت اصل توزیع عادلانه (ویژگی کارایی)، شاید تنها روشی باشد که بتواند در سناریوهای حساس و قانونی یک توضیح کامل و مستدل ارائه دهد.

همچنین این روش امکان تولید توضیحات متضاد^{۱۳۴} را فراهم می‌آورد. با این وجود، محاسبه مقادیر شاپلی از نظر پردازشی به شدت پرهزینه و زمان‌بر است و در اکثر مسائل دنیای واقعی تنها می‌توان به راه‌حل‌های تقریبی اکتفا کرد.

مشکل دیگر، احتمال تفسیر اشتباه این مقادیر است؛ مقدار شاپلی نشان‌دهنده تفاوت پیش‌بینی در صورت «حذف کامل ویژگی از آموزش مدل» نیست، بلکه مشارکت آن را در قالب ائتلاف‌های فعلی نشان می‌دهد.

علاوه بر این، این روش توضیحات خلاصه^{۱۳۵} تولید نمی‌کند و همواره سهمی را به تمام ویژگی‌ها اختصاص می‌دهد که ممکن است برای کاربران غیرمتخصص پیچیده باشد.

در نهایت، مشابه بسیاری از روش‌های مبتنی بر جایگشت، چنانچه ویژگی‌ها در مجموعه داده با یکدیگر همبستگی^{۱۳۶} داشته باشند، فرآیند نمونه‌برداری حاشیه‌ای می‌تواند به تولید داده‌های مصنوعی نامحتمل و در نتیجه تخمین‌های غیرواقعی منجر شود.

۵-۴ اهمیت ویژگی (Feature Importance)

اهمیت یک ویژگی، برابر است با میزان افزایش در خطای پیش‌بینی مدل پس از اعمال جایگشت^{۱۳۷} (به هم‌ریختگی تصادفی) بر روی مقادیر آن ویژگی؛ عملی که باعث قطع شدن ارتباط بین ویژگی مذکور و خروجی واقعی می‌شود.

۱-۵-۴ مبانی نظری و الگوریتم

ایده اصلی این روش بسیار ساده و سراسر است: اهمیت یک ویژگی با اندازه‌گیری میزان افزایش خطای پیش‌بینی مدل پس از جایگشت مقادیر آن ویژگی سنجیده می‌شود.

یک ویژگی زمانی «مهم» تلقی می‌گردد که به هم‌ریختگی مقادیر آن باعث افزایش خطای مدل شود، زیرا این امر نشان می‌دهد که مدل برای انجام پیش‌بینی‌های خود به آن ویژگی متکی بوده است.

در مقابل، یک ویژگی «نامهم» است اگر جایگشت مقادیر آن تغییری در خطای مدل ایجاد نکند، چرا که در این حالت مدل اساساً آن ویژگی را در پیش‌بینی‌های خود نادیده گرفته است.

معیار اهمیت ویژگی مبتنی بر جایگشت نخستین بار توسط بریمن [۴] برای جنگل‌های تصادفی معرفی شد. بر اساس این ایده، فیشر و همکاران [۵] نسخه مستقل از مدلی^{۱۳۸} از اهمیت ویژگی را پیشنهاد دادند و آن را «اتکای مدل»^{۱۳۹} نامیدند.

¹³⁴Contrastive explanations

¹³⁵Sparse explanations

¹³⁶Correlation

¹³⁷Permutation

¹³⁸Model-agnostic

¹³⁹Model reliance

برای محاسبه این معیار بر اساس الگوریتم فیشر، با داشتن یک مدل آموزش دیده (f)، ماتریس ویژگی‌ها (X) و بردار هدف (y)، ابتدا خطای مدل اصلی برآورد می‌شود ($e_{orig} = L(y, f(X))$).

سپس به ازای هر ویژگی مانند j ، ماتریس ویژگی جدیدی (X_{perm}) با جایگشت مقادیر آن ویژگی ایجاد می‌گردد تا ارتباط بین آن و خروجی واقعی قطع شود.

پس از آن، خطای مدل بر روی این داده‌های جایگشت‌یافته مجدداً محاسبه می‌گردد ($e_{perm} = L(y, f(X_{perm}))$). در نهایت، اهمیت ویژگی (FI_j) از طریق محاسبه نسبت این دو خطا ($FI_j = e_{perm}/e_{orig}$) یا تفاضل آن‌ها ($FI_j = e_{perm} - e_{orig}$) به دست آمده و ویژگی‌ها بر اساس این مقادیر به صورت نزولی مرتب می‌شوند.

۴-۵-۲ بررسی مجموعه داده مناسب برای محاسبه اهمیت ویژگی

پاسخ به این پرسش که آیا اهمیت ویژگی باید روی داده‌های آموزشی^{۱۴۰} محاسبه شود یا داده‌های آزمون^{۱۴۱}، مستقیماً به درک ما از ماهیت و تعریف دقیق «اهمیت ویژگی» بستگی دارد. بهترین راه برای درک این تفاوت، بررسی یک مثال است: فرض کنید یک مدل ماشین بردار پشتیبان^{۱۴۲} برای پیش‌بینی یک خروجی تصادفی و پیوسته، بر اساس ۵۰ ویژگی کاملاً تصادفی (و بی‌ارتباط با خروجی) آموزش داده شده است. در این حالت، مدل روی داده‌های آموزشی به شدت دچار بیش‌برازش^{۱۴۳} می‌شود؛ به طوری که خطای مطلق میانگین^{۱۴۴} آن روی داده‌های آموزشی 0.29 و روی داده‌های آزمون 0.82 خواهد بود (یعنی مدلی کاملاً ناکارآمد است).

اکنون پرسش این است: اهمیت این ۵۰ ویژگی چقدر باید باشد؟ آیا باید برابر با صفر باشد (زیرا در عمل هیچ‌یک کمکی به پیش‌بینی روی داده‌های نادیده نمی‌کنند)، یا باید صرفاً نشان‌دهنده میزان اتکای مدل به آن‌ها باشد (بدون در نظر گرفتن قابلیت تعمیم‌پذیری مدل)؟

استدلال طرفداران استفاده از «داده‌های آزمون» این است که تخمین خطای مدل روی داده‌های آموزشی، همواره خوش‌بینانه و گمراه‌کننده است. بنابراین، از آنجا که شاخص اهمیت ویژگی مبتنی بر خطای مدل است، محاسبه آن روی داده‌های آموزشی باعث می‌شود به اشتباه تصور کنیم ویژگی‌ها مهم هستند؛ در حالی که مدل صرفاً نویزها را حفظ کرده و بیش‌برازش کرده است.

در مقابل، طرفداران استفاده از «داده‌های آموزشی» استدلال می‌کنند که محاسبه اهمیت ویژگی روی این داده‌ها، دقیقاً نشان می‌دهد که مدل در عمل برای تصمیم‌گیری‌های خود تا چه حد به یک ویژگی اتکا کرده است (مشابه مفهومی که نمودارهای وابستگی جزئی^{۱۴۵} به تصویر می‌کشند). علاوه بر این، از منظر عملی، پژوهشگران معمولاً تمایل دارند از تمامی داده‌های در دسترس خود برای آموزش نهایی مدل استفاده کنند؛ بنابراین، کنار گذاشتن بخشی از داده‌ها صرفاً با هدف محاسبه اهمیت ویژگی، ممکن است رویکردی بهینه نباشد.

در نهایت، انتخاب میان این دو رویکرد به هدف اصلی شما از تفسیر مدل بستگی دارد: اگر می‌خواهید بدانید مدل برای «مکانیسم تصمیم‌گیری» خود چقدر به یک ویژگی تکیه کرده است، باید از داده‌های آموزشی استفاده کنید؛ اما اگر می‌خواهید بدانید یک ویژگی چقدر در «عملکرد و تعمیم‌پذیری مدل» روی داده‌های نادیده نقش دارد، باید به سراغ داده‌های آزمون

¹⁴⁰Training Data

¹⁴¹Test Data

¹⁴²Support Vector Machine (SVM)

¹⁴³Overfitting

¹⁴⁴Mean Absolute Error (MAE)

¹⁴⁵Partial Dependence Plots

بروید.

۳-۵-۴ مزایا و معایب

روش اهمیت ویژگی مبتنی بر جایگشت دارای مزایای یزرگی است. تفسیر این روش بسیار سراسر است، زیرا مستقیماً میزان افزایش خطا در صورت تخریب اطلاعات یک ویژگی را نشان می‌دهد.

این روش دیدگاهی سراسری^{۱۴۶} و به شدت فشرده از رفتار مدل ارائه می‌دهد و استفاده از نسبت خطا (به جای تفاضل) امکان مقایسه نتایج را در مسائل مختلف فراهم می‌سازد. همچنین، با جایگشت یک ویژگی، اثرات تعاملی^{۱۴۷} آن با سایر ویژگی‌ها نیز از بین می‌رود، که این یعنی روش مذکور به طور خودکار هم اثر اصلی و هم اثرات تعاملی را در نظر می‌گیرد. مزیت مهم دیگر این است که برخلاف برخی روش‌ها، در اینجا هیچ نیازی به آموزش مجدد^{۱۴۸} مدل وجود ندارد که این امر باعث صرفه‌جویی چشمگیری در زمان و توان محاسباتی می‌شود.

از سوی دیگر، این روش با معایب و چالش‌هایی نیز روبه‌روست. عدم قطعیت در انتخاب بین داده‌های آموزشی و آزمون یکی از چالش‌های بنیادین آن است. همچنین این روش کاملاً به خطای مدل گره خورده است؛ در حالی که گاهی پژوهشگران بیشتر به میزان تغییرپذیری و واریانس خروجی مدل (فارغ از عملکرد آن) علاقه‌مند هستند. علاوه بر این، محاسبه این معیار مستلزم دسترسی به برچسب‌های واقعی^{۱۴۹} داده‌هاست. ماهیت تصادفی جایگشت نیز باعث می‌شود که نتایج در هر بار اجرا کمی متفاوت باشند، که برای پایداری آن نیاز به تکرار محاسبات و میانگین‌گیری است.

در نهایت، یکی از بزرگ‌ترین چالش‌ها، مواجهه با ویژگی‌های همبسته^{۱۵۰} است. جایگشت در ویژگی‌های همبسته باعث تولید نقاط داده‌ای غیرمنطقی و غیرمحمول در دنیای واقعی می‌شود (مثلاً فردی با قد ۲ متر و وزن ۳۰ کیلوگرم) که ارزیابی مدل روی چنین داده‌هایی می‌تواند نتایج را دچار سوگیری کند. همچنین افزودن یک ویژگی همبسته می‌تواند اهمیت ویژگی مرتبط قبلی را کاهش داده و اهمیت کل را بین آن دو تقسیم نماید که این امر تفسیر نتایج را به شدت پیچیده می‌کند.

¹⁴⁶Global insight

¹⁴⁷Interaction effects

¹⁴⁸Retraining

¹⁴⁹True outcome

¹⁵⁰Correlated features

۵ توضیحات مبتنی بر نمونه

۱-۵ نمونه‌های اولیه و استثناها (Prototypes and Criticisms)

یک «نمونه اولیه»^{۱۵۱}، نقطه داده‌ای است که نماینده تمام داده‌ها محسوب می‌شود.

در مقابل، یک «استثنا»^{۱۵۲} نقطه داده‌ای است که توسط مجموعه نمونه‌های اولیه به خوبی نمایندگی نمی‌شود. هدف از یافتن استثناها، ارائه بینش‌های تکمیلی در کنار نمونه‌های اولیه است، به ویژه برای نقاطی که مدل به درستی آن‌ها را نمایندگی نمی‌کند. این مفاهیم را می‌توان به صورت مستقل از مدل‌های یادگیری ماشین برای توصیف داده‌ها به کار برد، اما همچنین می‌توان از آن‌ها برای ایجاد یک مدل تفسیرپذیر یا افزایش قابلیت تفسیر یک مدل جعبه‌سیاه استفاده کرد.

در این مبحث، از اصطلاح «نقطه داده»^{۱۵۳} برای اشاره به یک نمونه منفرد استفاده می‌شود تا این مفهوم واضح گردد که هر نمونه در واقع یک نقطه در یک سیستم مختصات است که در آن هر ویژگی نمایانگر یک بُعد است.

روش‌های متعددی برای یافتن نمونه‌های اولیه وجود دارد؛ به عنوان مثال، الگوریتم خوشه‌بندی k-medoids که با الگوریتم k-means مرتبط است. به طور کلی، هر الگوریتم خوشه‌بندی که نقاط داده واقعی را به عنوان مرکز خوشه برمی‌گرداند، برای انتخاب نمونه‌های اولیه مناسب است. با این حال، اکثر این روش‌ها صرفاً نمونه‌های اولیه را پیدا می‌کنند و قادر به یافتن استثناها نیستند.

روش MMD-critic که توسط کیم و همکاران [۶] معرفی شد، رویکردی است که هر دو مفهوم نمونه اولیه و استثنا را در یک چارچوب واحد ترکیب می‌کند. این روش توزیع داده‌ها را با توزیع نمونه‌های اولیه انتخاب‌شده مقایسه می‌کند. روش MMD-critic نمونه‌های اولیه‌ای را انتخاب می‌کند که تضاد میان این دو توزیع را به حداقل برسانند. نقاط داده در مناطقی با چگالی بالا، نمونه‌های اولیه مناسبی هستند. در مقابل، نقاط داده در مناطقی که توسط نمونه‌های اولیه به خوبی توضیح داده نمی‌شوند، به عنوان استثنا انتخاب می‌گردند.

۱-۱-۵ تئوری MMD-critic

رویه اجرای MMD-critic را می‌توان به طور خلاصه در سه گام بیان کرد: نخست، تعیین تعداد نمونه‌های اولیه و استثنای مورد نیاز؛ دوم، یافتن نمونه‌های اولیه با استفاده از یک جستجوی حریصانه^{۱۵۴} به گونه‌ای که توزیع آن‌ها به توزیع کل داده‌ها نزدیک باشد؛ و سوم، یافتن استثناها با جستجوی حریصانه در نقاطی که توزیع نمونه‌های اولیه با توزیع داده‌ها تفاوت دارد.

برای پیاده‌سازی این روش، به چند ابزار پایه نیاز است. اولین ابزار، یک تابع هسته^{۱۵۵} برای تخمین چگالی داده‌هاست. تابع هسته بر اساس میزان نزدیکی دو نقطه داده، به آن‌ها وزن می‌دهد. بر اساس این تخمین‌های چگالی، به معیاری نیاز داریم تا تفاوت دو توزیع را بسنجد. این مسئله با اندازه‌گیری «حداکثر میانگین تفاوت» (MMD)^{۱۵۶} حل می‌شود. مربع این معیار

¹⁵¹Prototype

¹⁵²Criticism

¹⁵³Data point

¹⁵⁴Greedy search

¹⁵⁵Kernel function

¹⁵⁶Maximum Mean Discrepancy

(MMD2) به صورت زیر محاسبه می‌گردد:

$$MMD^2 = \frac{1}{m^2} \sum_{i,j=1}^m k(z_i, z_j) - \frac{2}{mn} \sum_{i,j=1}^{m,n} k(z_i, x_j) + \frac{1}{n^2} \sum_{i,j=1}^n k(x_i, x_j)$$

در این رابطه، k تابع هسته، m تعداد نمونه‌های اولیه (z) و n تعداد کل نقاط داده (x) است. هدف الگوریتم، به حداقل رساندن MMD^2 است؛ هرچه این مقدار به صفر نزدیک‌تر باشد، توزیع نمونه‌های اولیه با تقریب بهتری داده‌ها را پوشش می‌دهد. یکی از توابع هسته متداول، هسته پایه شعاعی^{۱۵۷} است:

$$k(x, x') = \exp(-\gamma \|x - x'\|^2)$$

که در آن $\|x - x'\|^2$ فاصله اقلیدسی بین دو نقطه و γ یک پارامتر مقیاس‌دهی است.

۲-۱-۵ یافتن استثناها و تابع شاهد

برای یافتن استثناها، به ابزاری به نام تابع شاهد نیاز است که به ما نشان می‌دهد دو تخمین چگالی در یک نقطه خاص چقدر با هم تفاوت دارند. این تابع به صورت زیر تخمین زده می‌شود:

$$\text{witness}(x) = \frac{1}{n} \sum_{i=1}^n k(x, x_i) - \frac{1}{m} \sum_{j=1}^m k(x, z_j)$$

این تابع ارزیابی می‌کند که یک نقطه x در کدام توزیع تجربی بهتر جای می‌گیرد. برای یافتن استثناها، به دنبال مقادیر افراطی (بسیار مثبت یا بسیار منفی) تابع شاهد هستیم. اگر تابع شاهد برای نقطه‌ای نزدیک به صفر باشد، به این معناست که توزیع نمونه‌های اولیه و داده‌ها در آن نقطه هم‌خوانی دارند. یک مقدار مثبت بزرگ نشان‌دهنده آن است که توزیع نمونه‌های اولیه، توزیع داده‌ها را در آن نقطه بیش‌تخمین زده است؛ و برعکس، مقدار منفی بزرگ نشان می‌دهد که توزیع داده‌ها کم‌تخمین زده شده است (مثلاً داده‌های زیادی وجود دارند اما نمونه اولیه‌ای برای آن‌ها انتخاب نشده است). مانند نمونه‌های اولیه، استثناها نیز از طریق جستجوی حریصانه یافت می‌شوند، اما در اینجا هدف بهینه‌سازی یک تابع هزینه است که شامل تابع شاهد و یک ترم تنظیم‌کننده^{۱۵۸} برای ایجاد تنوع در نقاط انتخابی است.

۳-۱-۵ کاربردهای MMD-critic در یادگیری ماشین تفسیرپذیر

الگوریتم MMD-critic از سه طریق به ارتقای تفسیرپذیری در یادگیری ماشین کمک می‌کند.

نخست اینکه، این الگوریتم موجب درک بهتر توزیع داده‌ها می‌شود؛ قابلیت‌ای که به‌ویژه در مواجهه با توزیع‌های پیچیده و موارد مرزی بسیار ارزشمند است.

کاربرد دوم آن، کمک به ایجاد مدل‌های پیش‌بینی تفسیرپذیر است. به عنوان مثال، در «مدل نزدیک‌ترین نمونه اولیه»^{۱۵۹}، پیش‌بینی برای هر داده‌ی جدید صرفاً بر اساس نزدیک‌ترین نمونه‌ی اولیه به آن و از طریق رابطه $\hat{f}(x) = \arg \max_{i \in S} k(x, x_i)$ انجام می‌شود.

¹⁵⁷Radial basis function kernel

¹⁵⁸Regularizer term

¹⁵⁹Nearest Prototype Model

در نهایت، رویکرد سوم به تفسیرپذیر کردن مدل‌های جعبه‌سیاه اختصاص دارد. در این حالت، با استخراج نمونه‌های اولیه و استثناها و سپس بررسی پیش‌بینی‌های مدل برای این نقاط کلیدی، می‌توان نقاط ضعف پنهان مدل را شناسایی کرد. این رویکرد به عنوان یک آزمون صحت‌سنجی^{۱۶۰} برای کشف خطاهای سیستماتیک (مانند بروز سوگیری در دسته‌بندهای تصویر) بسیار کارآمد عمل می‌کند؛ زیرا اگرچه بررسی دستی و تک‌به‌تک تمام داده‌های آموزشی عملاً غیرممکن است، اما تحلیل و واکاوی دقیق چند نمونه اولیه و استثنا کاملاً عملی و راهگشا خواهد بود.

۲-۵ توضیحات خلاف واقع (Counterfactual Explanations)

یک توضیح خلاف واقع، یک وضعیت علی^{۱۶۱} را به این شکل توصیف می‌کند: «اگر X رخ نمی‌داد، Y نیز به وقوع نمی‌پیوست». به عنوان مثال: «اگر از این قهوه داغ نمی‌نوشیدم، زبانم نمی‌سوخت». تفکر به صورت خلاف واقع نیازمند تصور یک واقعیت فرضی است که با حقایق مشاهده‌شده در تناقض باشد (مانند دنیایی که در آن قهوه داغ نوشیده نشده است) و به همین دلیل به آن «خلاف واقع» می‌گویند.

در زمینه یادگیری ماشین تفسیرپذیر، می‌توان از توضیحات خلاف واقع برای توضیح پیش‌بینی‌های نمونه‌های فردی استفاده کرد. در اینجا، «رویداد» همان خروجی پیش‌بینی شده برای یک نمونه است و «علل»، مقادیر خاص ویژگی‌های آن نمونه هستند که به مدل داده شده و آن پیش‌بینی مشخص را «موجب» شده‌اند.

حتی اگر در واقعیت رابطه بین ورودی‌ها و خروجی هدف یک رابطه علی نباشد، ما می‌توانیم ورودی‌های یک مدل را به عنوان علت پیش‌بینی آن در نظر بگیریم. با تغییر دادن مقادیر ویژگی‌های یک نمونه پیش از انجام پیش‌بینی، می‌توانیم نحوه تغییر خروجی را تحلیل کرده و سناریوهای خلاف واقع را شبیه‌سازی کنیم.

ما عموماً به دنبال سناریوهایی هستیم که در آن‌ها پیش‌بینی به شکل معناداری تغییر کند؛ مانند تغییر در کلاس پیش‌بینی شده (مثلاً تأیید یا رد شدن درخواست وام) یا رسیدن پیش‌بینی به یک آستانه خاص. بنابراین، یک توضیح خلاف واقع برای یک پیش‌بینی، کوچک‌ترین تغییر در مقادیر ویژگی‌ها را توصیف می‌کند که منجر به تغییر پیش‌بینی به یک خروجی از پیش تعریف شده می‌گردد. این روش به دلیل آنکه تنها با ورودی‌ها و خروجی‌های مدل سروکار دارد، یک روش مستقل از مدل^{۱۶۲} است.

بر خلاف نمونه‌های اولیه^{۱۶۳}، توضیحات خلاف واقع لزوماً نباید نمونه‌های واقعی از داده‌های آموزشی باشند، بلکه می‌توانند ترکیب جدیدی از مقادیر ویژگی‌ها را شکل دهند.

۱-۲-۵ معیارهای یک توضیح خلاف واقع مطلوب

توضیحات خلاف واقع به دلیل ماهیت متضاد و گزینشی خود، توضیحاتی بسیار کاربرپسند محسوب می‌شوند. با این وجود، این روش از پدیده‌ای به نام «اثر راشومون»^{۱۶۴} رنج می‌برد؛ پدیده‌ای که در آن چندین توضیح مختلف، همگی به یک اندازه در توصیف یک خروجی موفق هستند اما با یکدیگر در تناقض اند (برای یک نمونه واحد، چندین توضیح

¹⁶⁰Sanity Check

¹⁶¹Causal situation

¹⁶²Model-agnostic

¹⁶³Prototypes

¹⁶⁴Rashomon effect

خلاف واقع کاملاً متفاوت وجود دارد). برای مقابله با این تعدد حقایق و تعریف یک توضیح خلاف واقع خوب، معیارهایی وجود دارد. نخست، نمونه خلاف واقع باید پیش‌بینی از پیش‌تعریف شده را تا حد امکان با دقت تولید کند (یا حداقل به خروجی مطلوب بسیار نزدیک باشد). دوم، نمونه خلاف واقع باید از نظر مقادیر ویژگی‌ها تا حد امکان به نمونه اصلی شبیه باشد و کمترین تغییرات ممکن را اعمال کند که این امر نیازمند استفاده از یک معیار فاصله^{۱۶۵} مناسب مانند فاصله منهتن^{۱۶۶} است. معیار سوم این است که مقادیر ویژگی‌ها در نمونه خلاف واقع باید در دنیای واقعی محتمل و منطقی باشند (مثلاً پیشنهاد یک آپارتمان با مساحت منفی یا دارای ۲۰۰ اتاق، یک توضیح خلاف واقع بی‌ارزش است).

۵-۲-۲ تولید توضیحات خلاف واقع

یک رویکرد ساده و ابتدایی برای تولید توضیحات خلاف واقع، جستجو از طریق آزمون و خطا است. اما رویکردهای بهینه‌تری نظیر روش پیشنهادی واکتر و همکاران [۷] وجود دارند که بر پایه بهینه‌سازی یک تابع زیان^{۱۶۷} عمل می‌کنند. این تابع زیان به صورت زیر تعریف می‌شود:

$$L(x, x', y', \lambda) = \lambda \cdot (\hat{f}(x') - y')^2 + d(x, x')$$

در این معادله، جمله اول فاصله مجذور بین پیش‌بینی مدل برای نمونه خلاف واقع (x') و خروجی مطلوب (y') را نشان می‌دهد. جمله دوم، فاصله (d) بین نمونه اصلی (x) و نمونه خلاف واقع (x') است. پارامتر λ نیز تعادلی میان فاصله در پیش‌بینی و فاصله در مقادیر ویژگی‌ها ایجاد می‌کند. نویسندگان این روش پیشنهاد می‌کنند که به جای تعیین دقیق λ ، یک پارامتر تحمل^{۱۶۸} به نام ϵ برای میزان خطای مجاز در پیش‌بینی تعریف شود:

$$|\hat{f}(x') - y'| \leq \epsilon$$

برای بهینه‌سازی این تابع می‌توان از الگوریتم‌های استاندارد (مانند نلدِر-مید^{۱۶۹} یا الگوریتم‌های مبتنی بر گرادیان مانند ADAM) استفاده کرد تا در نهایت نمونه بهینه‌ای به دست آید که شرایط زیر را ارضا کند:

$$\arg \min_{x'} \max_{\lambda} L(x, x', y', \lambda)$$

تابع فاصله d بین نمونه اصلی و نمونه خلاف واقع، همان فاصله منهتن است که به صورت ویژگی محور و با معکوس انحراف مطلق از میانه^{۱۷۰} وزن‌دهی شده است:

$$d(x, x') = \sum_{j=1}^p \frac{|x_j - x'_j|}{MAD_j}$$

که در آن MAD به صورت زیر محاسبه می‌شود:

$$MAD_j = \text{median}_{i \in \{1, \dots, n\}} (|x_{i,j} - \text{median}_{l \in \{1, \dots, n\}} (x_{l,j})|)$$

استفاده از فاصله منهتن همراه با مقیاس‌بندی MAD نسبت به فاصله اقلیدسی ارجحیت دارد، زیرا پراکندگی^{۱۷۱} را پخش کرده و نسبت به داده‌های پرت^{۱۷۲} مقاوم‌تر است. رویه الگوریتم نیز بدین شکل است که ابتدا یک نمونه تصادفی به عنوان

¹⁶⁵Distance measure

¹⁶⁶Manhattan distance

¹⁶⁷Loss function

¹⁶⁸Tolerance

¹⁶⁹Nelder-Mead

¹⁷⁰Median Absolute Deviation (MAD)

¹⁷¹Sparsity

¹⁷²Outliers

خلاف واقع اولیه انتخاب می‌شود و سپس تابع زیان به طور مکرر با افزایش مقدار λ بهینه‌سازی می‌گردد تا زمانی که فاصله پیش‌بینی از خروجی مطلوب به کمتر از حد تحمل ϵ برسد.

۳-۲-۵ مثال‌های کاربردی

برای درک بهتر، دو مثال کاربردی از مطالعات واکتر و همکاران [۷] بررسی می‌شود. در مثال نخست، یک شبکه عصبی سه‌لایه برای پیش‌بینی معدل سال اول دانشجویان حقوق بر اساس معدل پیشین (GPA)، نمره آزمون ورودی (LSAT) و نژاد آموزش داده شده است (نمرات نرمال‌سازی شده‌اند، بنابراین نمره ۰ معادل میانگین است). هدف، یافتن تغییراتی در ویژگی‌های ورودی است که پیش‌بینی مدل را به مقدار ۰ (معادل عملکرد متوسط) برساند. نتایج نشان داد که برای دانشجویانی که مدل برای آن‌ها عملکردی پایین‌تر از میانگین پیش‌بینی کرده بود (نمرات منفی)، توضیحات خلاف واقع به‌منظور رساندن پیش‌بینی به سطح میانگین، همواره ویژگی نژاد را از سیاه‌پوست (کد ۱) به سفیدپوست (کد ۰) تغییر می‌داد. این موضوع بیانگر وجود سوگیری نژادی^{۱۷۳} در تصمیمات مدل است.

جدول ۱: نمونه‌هایی از توضیحات خلاف واقع برای مدل پیش‌بینی عملکرد دانشجویان حقوق

RACE x'	LSAT x'	GPA x'	RACE	LSAT	GPA	Score
0	34.0	3.1	0	39.0	3.1	0.17
0	32.4	3.7	0	48.0	3.7	0.54
0	33.5	3.3	1	28.0	3.3	-0.77
0	35.8	2.4	1	28.5	2.4	-0.83
0	34.9	2.7	0	18.3	2.7	-0.57

در مثال دوم، یک شبکه عصبی برای پیش‌بینی خطر ابتلا به دیابت بر اساس عواملی چون سن، شاخص توده بدنی (BMI) و غیره آموزش دیده است. توضیحات خلاف واقع برای تغییر امتیاز خطر به عدد 0.5 برای افراد مختلف، توصیه‌هایی دقیق ارائه داد؛ نظیر: «اگر سطح انسولین سرم ۲ ساعته شما 154.3 بود، نمره شما 0.51 می‌شد» یا «اگر غلظت گلوکز پلاسمای شما 158.3 و انسولین سرم شما 160.5 بود، نمره شما به 0.51 می‌رسید».

۴-۲-۵ مزایا و معایب

استفاده از توضیحات خلاف واقع با مزایا و محدودیت‌های مشخصی همراه است.

از نقطه نظر مزایا، تفسیر این رویکرد بسیار شفاف و بدون ابهام است؛ هیچ فرض پنهانی در کار نیست و بر خلاف روش‌هایی نظیر LIME، خطر برون‌یابی^{۱۷۴} نادرست مدل محلی وجود ندارد.

همچنین، این روش نیازی به دسترسی به داده‌های آموزشی یا ساختار درونی مدل ندارد و تنها با دسترسی به تابع پیش‌بینی (حتی از طریق یک وب API) قابل اجراست. این ویژگی برای شرکت‌هایی که قصد حفاظت از اسرار تجاری مدل‌ها و داده‌های خود را دارند، بسیار ایده‌آل است. علاوه بر این، روش مذکور نه تنها برای مدل‌های یادگیری ماشین، بلکه برای هر سیستم مبتنی بر قانون نیز کارآمد بوده و پیاده‌سازی آن از طریق بهینه‌سازی توابع زیان نسبتاً ساده است.

¹⁷³Racial Bias
¹⁷⁴Extrapolation

با این وجود، معایبی نیز وجود دارد. مهم‌ترین چالش، اثر راشومون یا وجود چندین توضیح خلاف‌واقع برای یک نمونه است که انتخاب بهترین توضیح را به یک مسئله طراحی در هر پروژه تبدیل می‌کند. علاوه بر این، هیچ تضمین ریاضیاتی وجود ندارد که الگوریتم همواره برای یک حد تحمل^{۱۷۵}، مشخص، حتماً یک نمونه خلاف‌واقع پیدا کند. چالش دیگر، ضعف این روش در مدیریت ویژگی‌های رسته‌ای^{۱۷۶} با سطوح بسیار متنوع است. اجرای روش برای تمام ترکیبات ممکن از ویژگی‌های طبقه‌ای منجر به انفجار ترکیبیاتی^{۱۷۷} می‌شود (مثلاً ۶ ویژگی با ۱۰ سطح، نیازمند یک میلیون بار اجرای الگوریتم است). در نهایت، فقدان یک پیاده‌سازی نرم‌افزاری استاندارد و جامع برای این روش تا به امروز، یکی دیگر از محدودیت‌های کاربردی آن محسوب می‌شود.

۳-۵ نمونه‌های متخاصم (Adversarial Examples)

یک نمونه متخاصم، نقطه داده‌ای است که شامل تغییرات و اعوجاج‌های^{۱۷۸} جزئی و عمدی در ویژگی‌هایش است تا مدل یادگیری ماشین را فریب داده و آن را وادار به یک پیش‌بینی کاملاً اشتباه کند.

این مفهوم شباهت بسیاری به توضیحات خلاف‌واقع^{۱۷۹} دارد، با این تفاوت اساسی که هدف نمونه‌های متخاصم فریب دادن مدل است و نه تفسیر آن.

این نمونه‌ها صرفاً خروجی‌های جانبی مدل‌های یادگیری ماشین نیستند، بلکه پیامدهای کاربردی و امنیتی بسیار مهمی دارند و مدل‌ها را در برابر حملات آسیب‌پذیر می‌سازند. به عنوان مثال، یک خودروی خودران ممکن است به دلیل وجود یک برچسب کوچک روی تابلوی ایست که برای چشم انسان تنها شبیه به کمی کثیفی است اما سیستم بینایی ماشین آن را تابلوی توقف ممنوع تشخیص می‌دهد، دچار تصادف شود. به طور مشابه، فیلترهای هرزنامه^{۱۷۹} ممکن است فریب ایمیل‌هایی را بخورند که با دقت برای دور زدن الگوریتم‌ها طراحی شده‌اند، یا اسکنرهای امنیتی فرودگاه ممکن است یک سلاح دستکاری‌شده را به عنوان چتر طبقه‌بندی کنند.

۱-۳-۵ روش‌ها و مثال‌های تولید نمونه‌های متخاصم

تکنیک‌های متعددی برای ایجاد نمونه‌های متخاصم وجود دارد.

رویکرد غالب در اکثر این روش‌ها، کمینه‌سازی فاصله بین نمونه متخاصم و نمونه اصلی است، در حالی که پیش‌بینی مدل به سمت خروجی (متخاصم) دلخواه سوق داده می‌شود. برخی از این روش‌ها نیازمند دسترسی به گرادینت‌های^{۱۸۰} مدل هستند (که تنها برای مدل‌های مبتنی بر گرادینت مانند شبکه‌های عصبی کاربرد دارد)، در حالی که برخی دیگر تنها به تابع پیش‌بینی نیاز داشته و در نتیجه مستقل از مدل^{۱۸۱} هستند.

در حوزه بینایی ماشین، نمونه‌های متخاصم تصاویری هستند که پیکسل‌های آن‌ها به طور عمدی تغییر یافته‌اند و به طرز شگفت‌انگیزی نشان می‌دهند که چگونه شبکه‌های عصبی عمیق می‌توانند توسط تصاویری که برای چشم انسان کاملاً عادی به نظر می‌رسند، فریب بخورند.

¹⁷⁵Categorical features

¹⁷⁶Combinatorial explosion

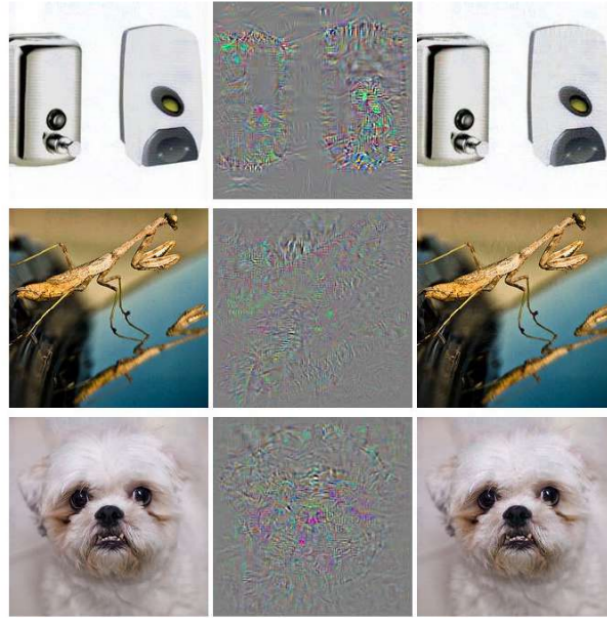
¹⁷⁷Perturbations

¹⁷⁸Counterfactual Explanations

¹⁷⁹Spam detectors

¹⁸⁰Gradients

¹⁸¹Model-agnostic



شکل ۸: نمونه‌های خصمانه برای مدل AlexNET که توسط [۸] تولید شده‌اند. تمام تصاویر موجود در ستون سمت چپ توسط مدل به درستی طبقه‌بندی شده‌اند. ستون میانی، خطا یا اغتشاش افزوده شده به تصاویر را (به صورت بزرگ‌نمایی شده) نشان می‌دهد. با افزودن این اغتشاش به تصاویر اصلی، تصاویر ستون سمت راست ایجاد شده‌اند که همگی توسط مدل به اشتباه در دسته «شتر مرغ» (Ostrich) قرار گرفته‌اند.

رویکرد بهینه‌سازی و خطای جزئی

زگدی و همکاران [۸] از یک رویکرد بهینه‌سازی مبتنی بر گرادیان برای یافتن نمونه‌های متخاصم استفاده کردند. این نمونه‌ها با کمینه‌سازی تابع زیان زیر نسبت به متغیر r تولید می‌شوند:

$$\text{loss}(\hat{f}(x+r), l) + c \cdot |r|$$

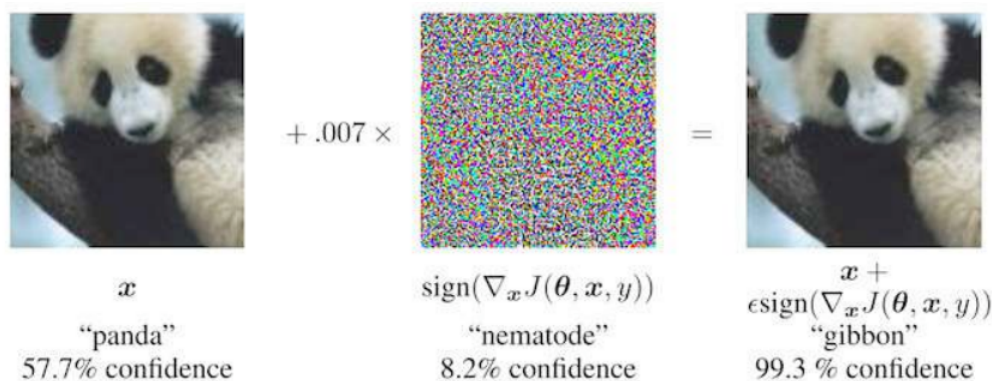
در این معادله، x تصویر اصلی (نمایش داده شده به صورت برداری از پیکسل‌ها)، r تغییرات اعمال شده بر پیکسل‌ها برای ایجاد تصویر متخاصم (به طوری که $x+r$ تصویر جدید را می‌سازد)، l کلاس خروجی مطلوبِ حمله، و پارامتر c برای ایجاد تعادل میان فاصله تصاویر و فاصله پیش‌بینی‌هاست.

جمله اول نشان‌دهنده فاصله بین پیش‌بینی نمونه متخاصم و کلاس هدف (l) بوده و جمله دوم فاصله بین نمونه متخاصم و تصویر اصلی را می‌سنجد. این فرمول‌بندی شباهت زیادی به توابع زیان در تولید توضیحات خلاف واقع دارد. این مسئله بهینه‌سازی با استفاده از الگوریتم L-BFGS (یک الگوریتم بهینه‌سازی مبتنی بر گرادیان) و با اعمال قیدهایی برای نگه داشتن مقادیر پیکسل‌ها بین ۰ و ۱ حل می‌شود.

روش علامت گرادیان سریع (Fast Gradient Sign Method / FGSM)

گودفلو و همکاران [۹] روش علامت گرادیان سریع را برای تولید تصاویر متخاصم ابداع کردند. این روش با افزودن یا کسر کردن یک خطای بسیار کوچک ϵ از هر پیکسل تصویر اصلی x عمل می‌کند. اینکه این مقدار اضافه یا کم شود، کاملاً به علامت گرادیان در آن پیکسل بستگی دارد:

$$x' = x + \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$$



شکل ۹: Goodfellow و همکاران [۹] نشان دادند که می‌توان یک تصویر پاندا را برای شبکه عصبی شبیه گیبون (Gibbon) کرد. با افزودن اغتشاش‌های کوچک (تصویر وسط) به پیکسل‌های تصویر اصلی پاندا (تصویر چپ)، نویسندگان یک نمونه خصمانه ایجاد کردند که توسط مدل به عنوان گیبون طبقه‌بندی می‌شود (تصویر راست)، در حالی که برای انسان همچنان شبیه یک پاندا به نظر می‌رسد.

در این رابطه، $\nabla_x J$ گرادیان تابع زیان مدل نسبت به بردار پیکسل‌های ورودی، y بردار برچسب واقعی و θ بردار پارامترهای مدل است. افزودن خطا در جهت گرادیان به این معناست که تصویر به گونه‌ای تغییر می‌یابد که خطای مدل (زیان) به حداکثر برسد. این آسیب‌پذیری زمانی رخ می‌دهد که شبکه عصبی، رابطه بین شدت روشنایی یک پیکسل و امتیاز کلاس را به صورت خطی پردازش کند. در نتیجه، مدل‌هایی که ساختارهای خطی را دنبال می‌کنند (مانند توابع فعال‌سازی ReLU یا رگرسیون لجستیک)، به شدت در برابر این روش آسیب‌پذیرند. این حمله با استفاده از مفهوم برون‌یابی^{۱۸۲} انجام می‌شود و مقادیر پیکسل‌ها را به مناطقی خارج از توزیع داده‌های طبیعی سوق می‌دهد.

حمله تک‌پیکسلی (1-pixel attack)

در حالی که روش گودفلو نیازمند تغییرات (هرچند جزئی) در بسیاری از پیکسل‌هاست، سو و همکاران [۱۰] نشان دادند که فریب دادن مدل تنها با تغییر دادن هدفمند یک پیکسل نیز امکان‌پذیر است. حمله تک‌پیکسلی از الگوریتم تکامل تفاضلی^{۱۸۳} برای یافتن پیکسل بهینه و نحوه تغییر رنگ آن استفاده می‌کند. در این روش، جمعیتی از راه‌حل‌های کاندید (که هر یک شامل مختصات مکانی و مقادیر RGB یک پیکسل است) به صورت مکرر با استفاده از فرمول زیر ترکیب و تکامل می‌یابند:

$$x_i(g+1) = x_{r_1}(g) + F \cdot (x_{r_2}(g) + x_{r_3}(g))$$

که در آن x_i عنصری از راه‌حل کاندید، g شماره نسل فعلی، F پارامتر مقیاس‌دهی و r_1, r_2, r_3 اعداد تصادفی متفاوتی هستند. این فرآیند تا زمانی که مدل فریب بخورد یا به حداکثر تعداد تکرار برسد، ادامه می‌یابد.

وصله متخاصم (Adversarial patch)

براون و همکاران [۱۱] با طراحی یک برچسب قابل چاپ، نمونه‌های متخاصم را وارد دنیای فیزیکی کردند. در این روش، شرط نزدیکی و شباهت بسیار زیاد تصویر متخاصم به تصویر اصلی حذف شده و به جای آن، بخشی از تصویر کاملاً با

¹⁸²Extrapolation

¹⁸³Differential evolution



شکل ۱۰: با تغییر عمده‌ی تنها یک پیکسل (که با دایره‌های قرمز مشخص شده است)، یک شبکه عصبی آموزش دیده روی ImageNet فریب داده می‌شود تا به جای کلاس اصلی (نوشته شده با رنگ مشکی)، کلاس اشتباه (نوشته شده با رنگ آبی) را پیش‌بینی کند. این کار توسط سو و همکاران [۱۰] انجام شده است.

یک «وصله» جایگزین می‌شود. الگوریتم تولید این وصله بر روی پس‌زمینه‌ها، موقعیت‌ها، زوایا و ابعاد مختلفی بهینه‌سازی می‌شود تا این برچسب بتواند در دنیای واقعی و در شرایط متنوع، سیستم‌های بینایی ماشین را فریب داده و به عنوان مثال یک موز را به عنوان یک توستر^{۱۸۴} طبقه‌بندی کند.

نمونه‌های متخاصم مقاوم در فضای سه‌بعدی

آتالیه و همکاران [۱۲] این مفهوم را به سطح بالاتری ارتقا داده و یک شیء سه‌بعدی (مانند مجسمه یک لاک‌پشت) را چاپ کردند که از نظر شبکه‌های عصبی عمیق، از تقریباً هر زاویه‌ای یک اسلحه به نظر می‌رسید! روش‌های کلاسیک در مواجهه با تغییراتی نظیر چرخش یا تغییر زاویه دید شکست می‌خورند. برای غلبه بر این محدودیت، الگوریتم «امید ریاضی بر روی تبدیلات»^{۱۸۵} (EOT) معرفی شد که نمونه‌های متخاصم را بر روی توزیعی از تبدیلات ممکن بهینه‌سازی می‌کند. هدف این است که احتمال کلاس متخاصم (y_t) بر روی تمام تبدیلات ممکن به حداکثر برسد:

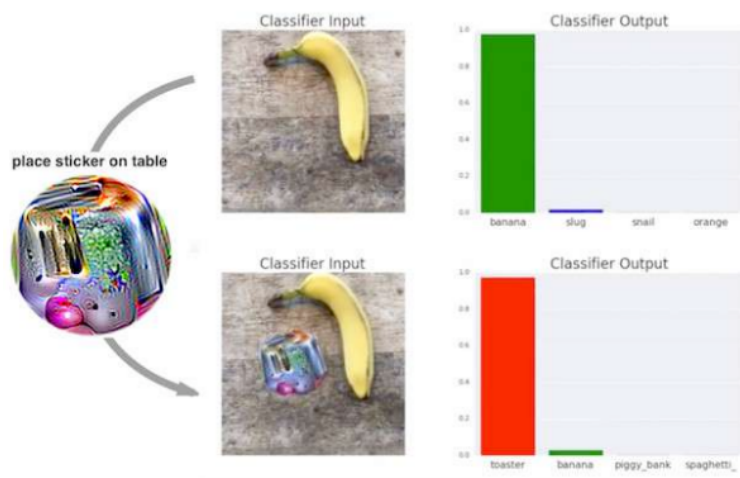
$$\arg \max_{x'} E_{t \sim T} [\log P(y_t | t(x'))]$$

در حالی که امید ریاضی فاصله بین تصویر متخاصم تبدیل یافته و تصویر اصلی تبدیل یافته زیر یک حد آستانه ϵ باقی بماند:

$$E_{t \sim T} [d(t(x'), t(x))] < \epsilon \quad \text{and} \quad x \in [0, 1]^d$$

¹⁸⁴Toaster

¹⁸⁵Expectation Over Transformation



شکل ۱۱: یک پتچ که باعث می شود یک طبقه بند VGG16 آموزش دیده روی ImageNET، تصویر یک موز را به اشتباه به عنوان توستر دسته بندی کند. این کار توسط براون و همکاران [۱۱] انجام شده است.

حمله جعبه سیاه (Black box attack)

در بسیاری از سناریوهای واقعی (مانند استفاده از یک Web API)، حمله کننده به پارامترها و گرادیان های مدل دسترسی ندارد.

پیپرنوت و همکاران [۱۳] روشی برای حملات جعبه سیاه (حملات با دانش صفر) ارائه کردند. این رویکرد با مجموعه ای از تصاویر از یک دامنه مشابه با داده های آموزشی آغاز می شود. پس از دریافت پیش بینی های مدل هدف (جعبه سیاه) برای این تصاویر، یک مدل جایگزین (مانند یک شبکه عصبی محلی) بر روی آن ها آموزش داده می شود. سپس، با استفاده از یک روش ابتکاری^{۱۸۶}، تصاویر مصنوعی جدیدی تولید می شوند تا مرزهای تصمیم گیری مدل بهتر شناسایی شوند و این فرآیند به صورت مکرر ادامه می یابد تا مدل جایگزین بتواند مرزهای مدل جعبه سیاه را تقریب بزند. در نهایت، با استفاده از روش هایی نظیر گرادیان سریع، نمونه های متخاصم برای مدل جایگزین تولید شده و از همان نمونه ها برای فریب دادن مدل اصلی جعبه سیاه استفاده می گردد. این روش به بسیار کارآمد است و حتی قابلیت فریب مدل های غیرعصبی (نظیر درختان تصمیم) را نیز دارد.

¹⁸⁶Heuristic

- [1] T. Miller, “Explanation in artificial intelligence: Insights from the social sciences,” *arXiv preprint arXiv:1706.07269*, 2017.
- [2] F. Doshi-Velez and B. Kim, “Towards a rigorous science of interpretable machine learning,” *arXiv preprint arXiv:1702.08608*, 2017.
- [3] L. S. Shapley, “A value for n-person games,” *Contributions to the Theory of Games*, vol. 2, no. 28, pp. 307–317, 1953.
- [4] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [5] A. Fisher, C. Rudin, and F. Dominici, “Model Class Reliance: Variable importance measures for any machine learning model class, from the ‘Rashomon’ perspective,” *arXiv preprint arXiv:1801.01489*, 2018.
- [6] B. Kim, R. Khanna, and O. O. Koyejo, “Examples are not enough, learn to criticize! Criticism for interpretability,” in *Advances in Neural Information Processing Systems (NeurIPS 2016)*, 2016.
- [7] S. Wachter, B. Mittelstadt, and C. Russell, “Counterfactual explanations without opening the black box: Automated decisions and the GDPR,” *arXiv preprint arXiv:1711.00399*, 2017.
- [8] C. Szegedy *et al.*, “Intriguing properties of neural networks,” *arXiv preprint arXiv:1312.6199*, 2013.
- [9] I. J. Goodfellow, J. Shlens, and C. Szegedy, “Explaining and harnessing adversarial examples,” *arXiv preprint arXiv:1412.6572*, 2014.
- [10] J. Su, D. V. Vargas, and K. Sakurai, “One pixel attack for fooling deep neural networks,” *IEEE Transactions on Evolutionary Computation*, 2019.
- [11] T. B. Brown *et al.*, “Adversarial patch,” *arXiv preprint arXiv:1712.09665*, 2017.
- [12] A. Athalye and I. Sutskever, “Synthesizing robust adversarial examples,” *arXiv preprint arXiv:1707.07397*, 2017.
- [13] N. Papernot *et al.*, “Practical black-box attacks against machine learning,” in *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security*, 2017.